

Санкт–Петербургский государственный университет
Факультет математики и компьютерных наук

ДРОЗДОВ Даниил Павлович

Выпускная квалификационная работа

***Применение алгоритмов машинного обучения и
эвристического поиска на графах Кэли в задаче сборки
кубиков Рубика большого размера***

Уровень образования: бакалавриат

Направление 01.03.02 «Прикладная математика и информатика»

Основная образовательная программа СВ.5156.2022 «Современное
программирование»

Научный руководитель:

Доцент, СПбГУ, к.ф.-м.н.,

Яковлев Константин Сергеевич

Рецензент:

Старший научный сотрудник, АНО

«Институт искусственного интел-
лекта», к.ф.-м.н.,

Андрейчук Антон Андреевич

Санкт-Петербург

2026

Содержание

Введение	3
1. Обзор предметной области	6
1.1. Методы на основе теории групп	6
1.2. Эвристический поиск с предварительно вычисленными эвристическими	7
1.3. Методы машинного обучения	7
2. Постановка задачи	11
3. Методы	14
3.1. Базовый алгоритм поиска кратчайшего пути	14
3.2. Эвристические алгоритмы поиска кратчайшего пути	17
3.3. Архитектуры нейросетей для предсказания эвристик	22
3.4. Алгоритмы генерации тренировочной выборки	28
4. Эксперименты	36
4.1. Эксперименты с обучающим датасетом	36
4.2. Эксперименты с архитектурами нейросетей	39
4.3. Эксперименты с алгоритмом поиска	42
4.4. Сводные выводы по экспериментам	44
4.5. Сравнение итогового метода с базовым подходом	45
Заключение	47
Список литературы	49

Введение

Задача поиска пути на графах большого размера является одной из центральных в прикладной математике и информатике и имеет широкий спектр практических приложений: планирование движений роботов [1], маршрутизация в сетях, биоинформатика (рис. 1), квантовые вычисления и др.. Особый класс таких задач связан с графами Кэли конечных групп — высокосимметричными графами, в которых вершины соответствуют элементам группы, а рёбра задаются действием фиксированного порождающего множества. Поиск кратчайшего пути на графах Кэли является NP-трудной задачей в общем случае [2], а размеры реальных групп, возникающих в приложениях, делают прямые методы перебора принципиально непригодными.

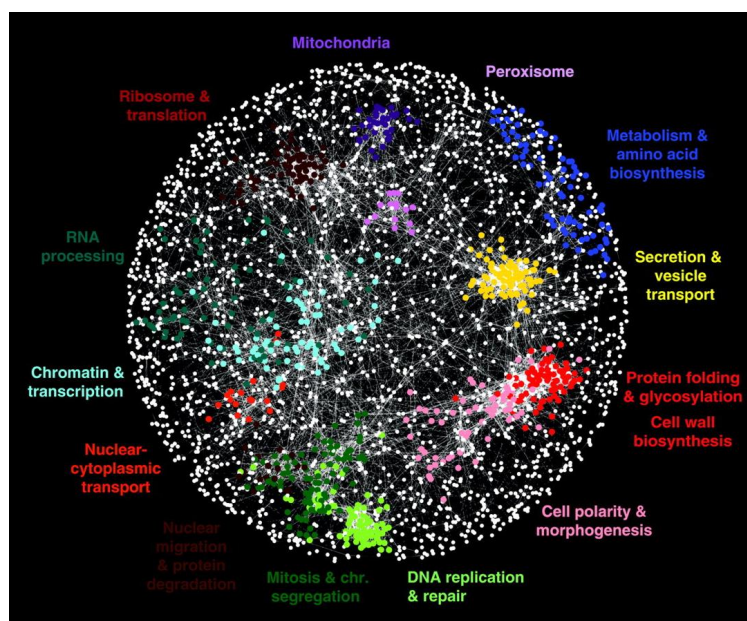


Рис. 1: Граф генов: вершины соответствуют генам, а рёбра отражают статистически значимое сходство их биологических профилей [4]

Кубик Рубика является каноническим примером такой задачи. Его состояния образуют граф Кэли (рис. 2), соответствующий группе симметрий, а сборка головоломки — поиск пути от произвольной вершины до единственной целевой. Для кубика $3 \times 3 \times 3$ задача хорошо изучена: пространство из $\approx 4,3 \times 10^{19}$ состояний позволяет применять методы с использованием pattern databases — заранее построенных таблиц, которые хранят оптимальные расстояния для отдельных частей кубика и тем самым ускоряют поиск

решения. Однако с ростом размера N ситуация кардинально меняется: группа кубика Рубика $4 \times 4 \times 4$ содержит $\approx 7,4 \times 10^{45}$ элементов, кубика $5 \times 5 \times 5$ — $\approx 1,2 \times 10^{74}$. При таких размерах классические алгоритмы на графах (BFS [19], A^* [21], IDA^* [22]) и системы компьютерной алгебры (GAP) полностью непригодны [5]: даже хранение таблицы состояний физически невозможно.

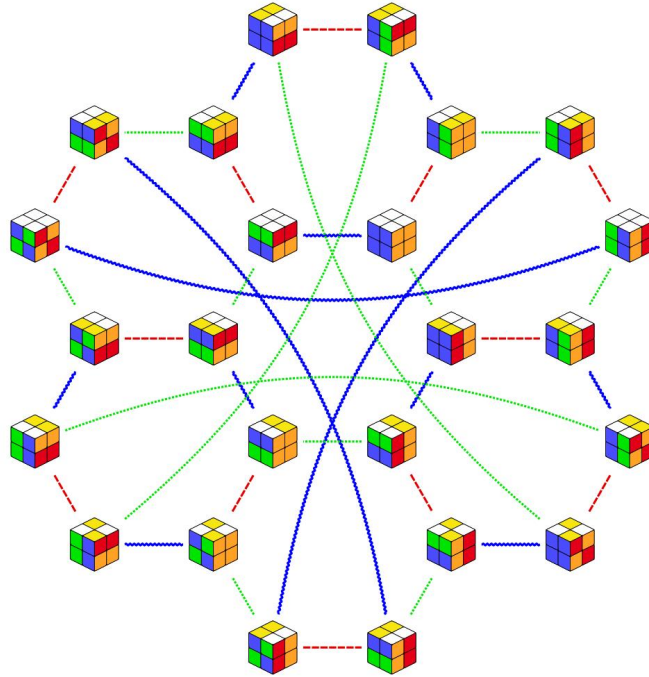


Рис. 2: Подграф графа Кэли кубика Рубика $2 \times 2 \times 2$ [6]

Это делает задачу сборки кубиков большого размера актуальным полигоном для разработки принципиально иных методов поиска. До 2025 года методы машинного обучения практически не применялись к кубикам крупнее $3 \times 3 \times 3$, а существующие нейросетевые подходы были сосредоточены главным образом на классическом кубике $3 \times 3 \times 3$. Наиболее известным методом являлся DeepCubeA [7], находивший решение оптимальной длины в 60,3% для $3 \times 3 \times 3$, тогда как его последователь EfficientCube [8] достигал 69,6% оптимальных решений. Соревнование Kaggle Santa 2023 [9], в котором участвовало более тысячи команд, продемонстрировало отсутствие эффективных ML-решений для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$.

В 2025 году был представлен метод [10], способный решать данную проблему и включающий три ключевых элемента: генерацию обучающих данных

через случайные блуждания по графу Кэли, обучение нейросети (ResMLP [11]) оценивать «близость» текущего состояния к целевому в смысле диффузионного расстояния (метрики, учитывающей множество возможных путей на графе состояний), а также многоагентный лучевой поиск (beam search) — алгоритм, который одновременно рассматривает несколько наиболее перспективных вариантов решения и постепенно отбирает лучшие. Метод стал первым ML-подходом, способным эффективно решать кубики $4 \times 4 \times 4$ и $5 \times 5 \times 5$, превзойдя все результаты соревнования Kaggle Santa 2023; при этом для $3 \times 3 \times 3$ 98,4% найденных им путей имели оптимальную длину. Вместе с тем в текущей реализации метод не демонстрирует обобщаемости за пределы кубика $5 \times 5 \times 5$: с ростом N резко возрастают требования к ширине луча и числу агентов, а качество решений ухудшается. Исследование и устранение этого ограничения представляют собой актуальную открытую задачу.

Целью данной работы является разработка метода поиска пути на больших графах Кэли с экспериментальным исследованием его применимости к кубикам Рубика размера $N \times N \times N$ при $N \geq 3$.

Для достижения цели решались следующие задачи:

1. Обзор существующих методов решения задачи сборки кубика Рубика и анализ их ограничений при росте N .
2. Изучение математической структуры группы кубика и её представления в виде графа Кэли.
3. Реализация генератора обучающих данных на основе случайных блужданий по графу Кэли.
4. Реализация и обучение нейросетевой модели для оценки расстояния произвольного состояния до целевого.
5. Реализация алгоритма эвристического поиска пути на графе.
6. Вычислительный эксперимент: оценка качества решений и сравнение с базовыми методами на кубиках $N \times N \times N$ при $N \geq 3$.

1. Обзор предметной области

Задача алгоритмического решения кубика Рубика изучается с начала 1980-х годов. За это время были предложены три принципиально различных класса подходов:

1. Методы, основанные на теории групп (описывающие преобразования кубика как элементы алгебраической структуры);
2. Методы эвристического поиска с использованием предварительно вычисленных эвристик;
3. Методы эвристического поиска с применением машинного обучения для построения оценочных функций.

В данном разделе рассматриваются ключевые работы каждого класса и анализируются причины, по которым ни один из существующих подходов не масштабируется на кубики размера $N > 5$.

1.1. Методы на основе теории групп

Первым алгоритмом, гарантирующим решение кубика $3 \times 3 \times 3$ за ограниченное число ходов, стал метод, предложенный в 1981 году [13], основанный на последовательном сведении задачи к более простым подзадачам. Его ключевая идея состоит в построении убывающей цепочки подгрупп $G_0 \supset G_1 \supset G_2 \supset G_3 \supset \{e\}$ группы кубика и последовательном переводе состояния из каждой подгруппы в следующую с помощью заранее вычисленных таблиц переходов. Алгоритм гарантировал решение не длиннее 52 ходов. В 1992 году был предложен двухфазный алгоритм [14] сократил цепочку до двух фаз, в котором решение разбивается на два этапа; это позволило находить решения длиной порядка 20 ходов за разумное время. Оба алгоритма существенно опираются на специфическую структуру группы кубика $3 \times 3 \times 3$ и требуют предварительного вычисления и хранения больших таблиц для каждой фазы. При переходе к кубикам большего размера объём этих таблиц растёт экспоненциально, что делает такие методы практически неприменимыми уже при $N \geq 4$.

1.2. Эвристический поиск с предварительно вычисленными эвристиками

В 1997 году были впервые получены оптимальные решения для случайных конфигураций кубика $3 \times 3 \times 3$ [15]. Предложенный метод основан на алгоритме IDA* (iterative deepening A*) с допустимой эвристикой, вычисляемой по заранее построенным базам подзадач (pattern databases). Такие базы представляют собой таблицы, в которых заранее записано минимальное число ходов, необходимое для приведения к целевому состоянию отдельных частей кубика (например, только углов или только рёбер). Комбинируя несколько таких баз и беря максимум соответствующих оценок, удалось получить достаточно точную нижнюю оценку расстояния до решения, что позволило алгоритму IDA* находить оптимальные решения за приемлемое время. Принципиальным ограничением данного подхода является экспоненциальный рост размера таких таблиц с увеличением размера кубика: уже для кубика $4 \times 4 \times 4$ соответствующие базы требуют недостижимых объёмов памяти (от десятков гигабайт до терабайтов даже для отдельных подзадач), что делает такие методы практически неприменимыми и оставляет задачу построения эффективных допустимых эвристик для $N \geq 4$ открытой.

1.3. Методы машинного обучения

Принципиально иной подход был предложен в работе DeepCubeA (Agostinelli et al., 2019) [7]. Авторы обучили глубокую нейронную сеть аппроксимировать функцию расстояния до целевого состояния методом обучения с подкреплением (автодидактическая итерация, ADI, рис. 3), в котором модель последовательно улучшает собственные оценки, используя сгенерированные ей же обучающие примеры. Полученная нейронная сеть используется для оценки эвристики — предсказания расстояния до целевого состояния, которая затем применяется в алгоритме BWAS (Batch Weighted A*) — варианте эвристического поиска, обрабатывающем состояния пакетами и ускоряющем исследование вершин графа. Метод был апробирован на кубике $3 \times 3 \times 3$ и показал оптимальность в 60,3% случаев на стандартном тестовом наборе. К недостаткам DeepCubeA относятся высокая вычислительная стоимость про-

цедуры ADI (требуется около 10 млрд обучающих примеров) и ограниченная оптимальность; кроме того, авторы не исследовали применимость метода к кубикам большего размера.

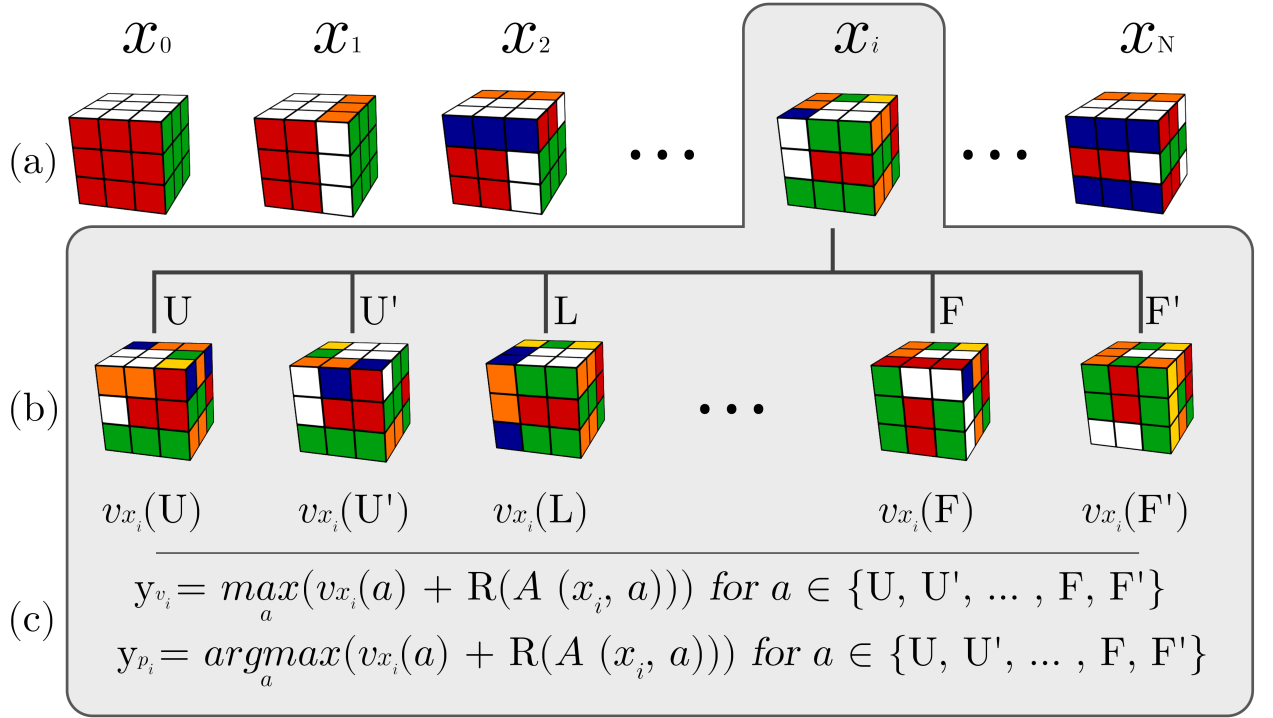


Рис. 3: Визуализация генерации тренировочного датасета с помощью автодидактической итерации [12]

Работа EfficientCube (Takano, 2023) [8] развивает идеи DeepCubeA, заменяя дорогостоящую процедуру ADI на самообучение (self-supervised learning): обучающие данные генерируются путём случайных перемешиваний из решённого состояния, а в качестве целевых меток используются последние выполненные ходы в этих последовательностях. В качестве алгоритма поиска используется лучевой поиск (beam search) — метод, который на каждом шаге сохраняет ограниченное число наиболее перспективных частичных решений и продлевает их параллельно, — что позволяет существенно ускорить нахождение решений по сравнению с BWAS. Метод достигает оптимальности 69,6% на том же наборе при существенно меньших затратах на обучение. Тем не менее EfficientCube также ограничен случаем кубика $3 \times 3 \times 3$: на больших кубиках метод не тестировался.

Соревнование Kaggle Santa 2023 [9], в котором участвовало более тысячи команд и ставилась задача решения виртуальных головоломок, включая

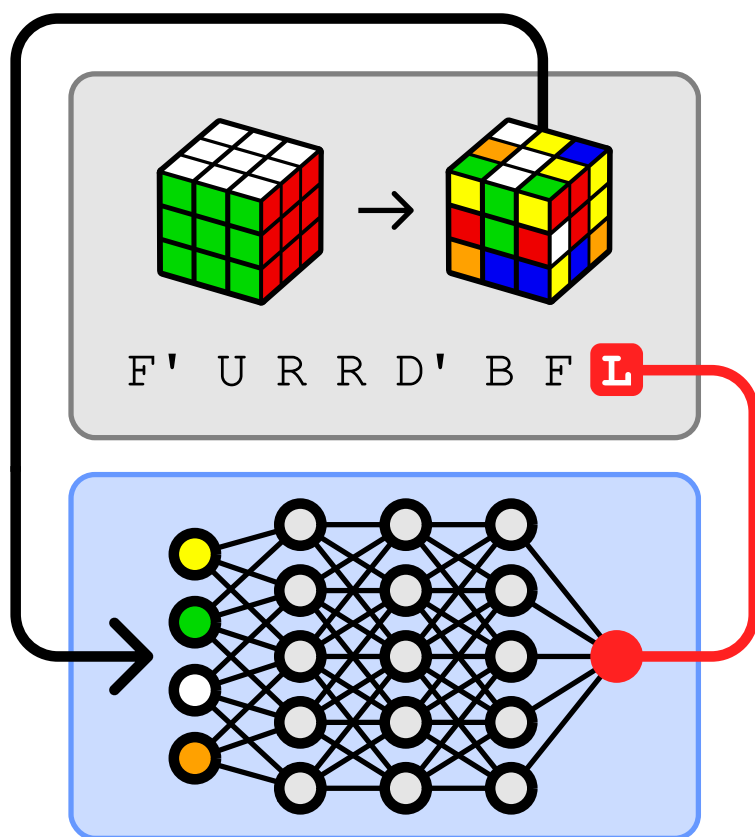


Рис. 4: Визуализация процесса self-supervised learning [8]

кубики $4 \times 4 \times 4$ и $5 \times 5 \times 5$, наглядно показало, что для больших кубиков эффективные решения, основанные на машинном обучении, по-прежнему отсутствуют: участники в основном использовали эвристические методы и специализированные алгоритмы сокращения пространства поиска, не опирающиеся на машинное обучение.

Первым ML-методом, успешно справившимся с решением кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$, стал подход CayleyPy (Chervov et al., NeurIPS 2025) [10]. Авторы отказались от дорогостоящих методов генерации обучающих данных, заменив их случайными блужданиями по графу Кэли: начиная из целевой вершины, выполняются случайные ходы и фиксируются пары (состояние, число ходов). Нейронная сеть обучается оценивать так называемое диффузионное расстояние — ожидаемое число ходов случайного блуждания, необходимое для достижения данного состояния. Полученная модель используется как эвристика в многоагентном лучевом поиске: обучается ансамбль из A независимых нейросетей, с каждой из которых запускается новый поиск, а итоговым решением выбирается кратчайший найденный путь. Метод превзошёл все ре-

зультаты Kaggle Santa 2023 для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$, а для кубика $3 \times 3 \times 3$ достиг оптимальности 98,4%, что сопоставимо со специализированными решателями на основе pattern databases. Вместе с тем авторы отмечают, что метод не обобщается за пределы $N = 5$: с ростом N требуемая ширина луча W и число агентов A растут слишком быстро, а качество решений существенно снижается.

Альтернативный подход к задаче поиска на графах был предложен в работе [16], опубликованной в 2026 году. Авторы применяют генеративные потоковые сети (Generative Flow Networks, GFlowNets [17]) для обучения модели, порождающей траектории в графе. В рамках этого подхода рассматриваются прямой и обратный потоки, задающие вероятностное распределение над путями в графе. Теоретически показано, что при минимизации несогласованности потока распределение траекторий концентрируется на кратчайших путях между начальным и целевым состояниями. На этой основе задача поиска пути формулируется как обучение GFlowNet с регуляризацией потока, при которой модель обучается согласовывать вероятности локальных переходов с глобальной структурой целевых траекторий. Экспериментально метод демонстрирует результаты, сопоставимые с DeepCubeA и EfficientCube на кубике $3 \times 3 \times 3$, при меньшем вычислительном бюджете на этапе тестирования. Однако, как и большинство предшествующих подходов, он не тестировался на кубиках размера $N \geq 4$.

2. Постановка задачи

Граф Кэли

Определение 1. Пусть G — конечная группа с единицей e , $S \subset G \setminus \{e\}$ — симметричное порождающее множество, то есть

$$S = S^{-1} \stackrel{def}{\iff} \forall s \in S : s^{-1} \in S.$$

Графом Кэли группы G с порождающим множеством S называется ориентированный граф

$$\text{Cay}(G, S) = (V, E), \quad V = G, \quad E = \{(g, g \cdot s) | g \in G, s \in S\}.$$

Граф $\text{Cay}(G, S)$ является $|S|$ -регулярным и вершинно-транзитивным: для любых $g, h \in G$ существует автоморфизм графа, переводящий g в h .

Пример 1. Граф Кэли $\Gamma(S_5, S)$ для группы S_5 — группы перестановок размера 5 с множеством генераторов $S = \{(1, 5, 4), (3, 4), (4, 5, 1)\}$ (рис. 5).

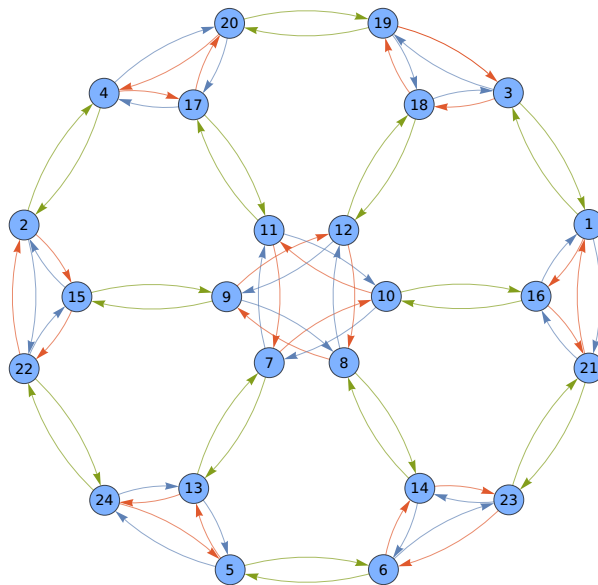


Рис. 5: Цветной ориентированный граф Кэли $\Gamma(S_5, S)$, где вершины — элементы группы, а окрашенные рёбра кодируют действие генераторов из множества S

Определение 2. Путём из вершины u в вершину v в графе $\text{Cay}(G, S)$ называется последовательность

$$\pi = (g_0, g_1, \dots, g_T), \quad g_0 = u, \quad g_T = v, \quad g_{k+1} = g_k \cdot s_k, \quad s_k \in S.$$

Число рёбер T называется длиной пути π .

Определение 3. Расстоянием между вершинами $u, v \in G$ называется

$$d(u, v) = \min\{T \mid \text{существует путь из } u \text{ в } v \text{ длины } T\}.$$

Диаметром графа Кэли называется

$$\text{diam}(G, S) = \max_{u, v \in G} d(u, v)$$

Группа кубика Рубика Кубик рубика $N \times N \times N$ имеет $6N^2$ facets. В данной работе используется метрика четвертьоборотов (quater-turn metric, QTM): каждый ход представляет собой поворот одного из $3N$ слоёв на 90 по часовой или против часовой стрелки. Зафиксируем нумерацию facets числами $\{0, 1, \dots, 6N^2 - 1\}$. Тогда каждый ход реализуется как перестановка

$$s : \{0, 1, \dots, 6N^2 - 1\} \rightarrow \{0, 1, \dots, 6N^2 - 1\}.$$

Определение 4. Группой кубика Рубика размера N называется группа

$$G_N = \langle S_N \rangle,$$

где S_N — множество всех элементарных ходов и их обратных, $|S_N| = 6N$. Единичный элемент $e \in G_N$ соответствует собранному состоянию кубика. Пространство состояний головоломки совпадает с $\text{Cay}(G_N, S_N)$.

Диаметр $\text{diam}(G_N, S_N)$ соответствует «числу Бога» — наименьшему числу ходов, достаточному для сборки из любого состояния. Мощности групп $|G_N|$ растут факториально с N ; известные значения приведены в таблице 1.

N	$ G_N $	$\text{diam}(G_N, S_N)$
2	$\approx 3.7 \times 10^6$	14
3	$\approx 4.3 \times 10^{19}$	26 [18]
4	$\approx 7.4 \times 10^{45}$	неизв.
5	$\approx 1.2 \times 10^{74}$	неизв.

Таблица 1: Мощности групп кубика Рубика и диаметры соответствующих графов Кэли в метрике QTM.

Представление состояния Каждому фацету кубика сопоставляется цвет соответствующей грани в собранном состоянии — число из множества $\{0, 1, 2, 3, 4, 5\}$. Тогда каждое состояние $g \in G$ однозначно задаётся вектором

$$v(g) = (c_0, c_1, \dots, c_{6N^2-1}) \in \{0, 1, 2, 3, 4, 5\}^{6N^2},$$

где c_i — цвет i -го фацета в состоянии g . Применение хода $s \in S_N$ к состоянию g даёт

$$v(g \cdot s) = v(g)[s^{-1}(0), s^{-1}(1), \dots, s^{-1}(6N^2 - 1)],$$

то есть переводит $v(g)$ в $v(g \cdot s)$ перестановкой координат согласно действию s . Это вычислительно дёшево и не требует хранения всего графа в памяти.

Задача Для произвольного состояния $g_0 \in G_N$ найти путь π^* из g_0 в e в графе $\text{Cay}(G_N, S_N)$ наименьшей длины, то есть

$$\pi^* = \arg \min_{\pi: g_0 \rightarrow e} |\pi|. \quad (1)$$

Решение задачи (1) является NP-трудным в общем случае [2] [3]. При $N \geq 4$ мощность $|G_N|$ такова, что применение точных алгоритмов поиска (BFS, IDA*) и систем компьютерной алгебры физически невозможно. Поэтому практически значимой является задача нахождения приближённого решения $\tilde{\pi}$ такого, что $|\tilde{\pi}|$ близко к $d(g_0, e)$. Качество приближённого решения оценивается следующими метриками:

Доля решённых: $\rho = \mathbb{P}[\tilde{\pi} \text{ найден за } I_{\max} \text{ итераций}];$

Средняя длина: $\bar{L} = \mathbb{E}[|\tilde{\pi}|].$

3. Методы

В данной главе рассматриваются методы решения задачи поиска пути на графах, а также архитектуры нейросетей и алгоритмы генерации обучающих выборок, использованные в рамках данной выпускной квалификационной работы.

В первой части приводится обзор алгоритмов поиска пути на графах и анализ их применимости к графам Кэли большого размера. Далее рассматриваются архитектуры нейросетей, используемых для оценки эвристик: анализируется их качество, а также вычислительная стоимость предсказания.

Алгоритмы генерации обучающих выборок рассматриваются в заключительном разделе главы.

3.1. Базовый алгоритм поиска кратчайшего пути

Алгоритмы поиска пути на графах, рассмотренные в данной главе, берут своё начало в базовом алгоритме Breadth-first search (поиск в ширину, BFS) [19]. Этот алгоритм решает задачу поиска кратчайшего пути между двумя вершинами в невзвешенном графе, то есть графе, в котором все рёбра имеют одинаковый вес, равный 1.

В ходе работы BFS строит дерево поиска. Каждая вершина этого дерева, называемая состоянием, сопоставляется с вершиной s исходного графа и хранит следующую информацию:

- ссылку на соответствующую вершину s исходного графа;
- ссылку на родительскую вершину, используемую для восстановления кратчайшего пути.

В процессе выполнения алгоритм последовательно расширяет дерево поиска. На каждой итерации $i \geq 0$ рассматривается множество вершин V_i , достижимых из начальной вершины за не более чем i шагов. Для каждой вершины $v \in V_i$ рассматриваются все её соседи, и формируется множество V_{i+1} , содержащее вершины, достижимые за не более чем $i + 1$ шагов. Если на некоторой итерации n целевая вершина впервые появляется в множестве V_n находится

целевая вершина, то кратчайший путь имеет длину n и восстанавливается по дереву поиска путём перехода по родительским ссылкам. Корректность следует из того, что все вершины, достижимые за менее чем n шагов, уже были рассмотрены на предыдущих итерациях. Также алгоритм можно моди-

Algorithm 1 Breadth-first search (BFS) как поиск в дереве состояний

Require: граф $G = (V, E)$, начальная вершина s , целевая вершина t

Ensure: кратчайший путь от s до t

```

1: создать начальное состояние  $x_0$ , соответствующее  $s$ 
2:  $parent[x_0] \leftarrow \text{nil}$ 
3:  $V_0 \leftarrow \{x_0\}$ 
4:  $i \leftarrow 0$ 
5: while  $V_i \neq \emptyset$  do
6:   for all  $x \in V_i$  do
7:     if вершина графа, соответствующая  $x$ , равна  $t$  then
8:       return вернуть путь, восстановленный по  $parent$ 
9:     end if
10:  end for
11:   $V_{i+1} \leftarrow \emptyset$ 
12:  for all  $x \in V_i$  do
13:     $v \leftarrow$  вершина графа, соответствующая  $x$ 
14:    for all  $u \in Adj(v)$  do
15:      создать новое состояние  $y$ , соответствующее  $u$ 
16:       $parent[y] \leftarrow x$ 
17:      добавить  $y$  в  $V_{i+1}$ 
18:    end for
19:  end for
20:   $i \leftarrow i + 1$ 
21: end while
22: return "пути не существует"
```

фицировать, поддерживая более сильный инвариант: множество V_i содержит все вершины, достижимые из начальной вершины ровно за i шагов. Для этого необходимо дополнительно хранить множество посещённых вершин и при построении множества V_{i+1} на шаге i исключать из него вершины, которые уже были достигнуты на предыдущих итерациях.

Преимуществом такой модификации является устранение повторного рассмотрения вершин, что уменьшает фактическое число обрабатываемых

состояний. В исходной формулировке без множества посещённых вершин число рассматриваемых состояний может расти экспоненциально, тогда как введение множества посещённых вершин ограничивает его линейной зависимостью от размера графа.

Algorithm 2 Breadth-first search (BFS) с инвариантом $\forall v \in V_i : d(s, v) = i$

Require: граф $G = (V, E)$, начальная вершина s , целевая вершина t

Ensure: кратчайший путь от s до t

```

1: создать начальное состояние  $x_0$ , соответствующее  $s$ 
2:  $parent[x_0] \leftarrow \text{nil}$ 
3:  $V_0 \leftarrow \{x_0\}$ 
4: отметить  $s$  как посещённую
5:  $i \leftarrow 0$ 
6: while  $V_i \neq \emptyset$  do
7:   for all  $x \in V_i$  do
8:     if вершина графа, соответствующая  $x$ , равна  $t$  then
9:       return путь, восстановленный по  $parent$ 
10:    end if
11:  end for
12:   $V_{i+1} \leftarrow \emptyset$ 
13:  for all  $x \in V_i$  do
14:     $v \leftarrow$  вершина графа, соответствующая  $x$ 
15:    for all  $u \in Adj(v)$  do
16:      if  $u$  не посещена then
17:        создать новое состояние  $y$ , соответствующее  $u$ 
18:         $parent[y] \leftarrow x$ 
19:        добавить  $y$  в  $V_{i+1}$ 
20:        отметить  $u$  как посещённую
21:      end if
22:    end for
23:  end for
24:   $i \leftarrow i + 1$ 
25: end while
26: return "пути не существует"

```

Несмотря на простоту и теоретическую корректность, алгоритм BFS (в том числе в модифицированной версии с множеством посещённых вершин) неприменим для графов Кэли большого размера. Это связано с тем, что число вершин такого графа совпадает с мощностью соответствующей группы и, как

правило, растёт экспоненциально с размером задачи. Например, для кубика $3 \times 3 \times 3$ число состояний порядка 10^{19} , и для кубиков ($N \geq 4$) оно становится существенно больше.

Алгоритм BFS требует хранения всех вершин текущего и предыдущих уровней поиска, что приводит к потреблению памяти порядка $O(|V|)$ и времени порядка $O(|E|)$. В условиях экспоненциального роста числа состояний это делает как исходную, так и модифицированную версии алгоритма практически неприменимыми.

Таким образом, для графов Кэли, возникающих в задачах наподобие кубика Рубика, требуется использование более сложных методов, основанных на эвристическом поиске, позволяющих ограничить рассматриваемое подмножество состояний.

3.2. Эвристические алгоритмы поиска кратчайшего пути

Best-first search

Ограничения алгоритма поиска в ширину при работе с графами большого размера приводят к необходимости использования более эффективных методов, способных направленно исследовать пространство состояний. Одним из таких подходов является класс алгоритмов best-first search (поиск «лучший — первый») [20].

В отличие от неэвристических методов, таких как BFS, алгоритмы best-first search используют дополнительную информацию о задаче в виде эвристической функции, позволяющей оценить «перспективность» текущего состояния. На каждой итерации из множества рассматриваемых состояний выбирается то, которое имеет наилучшее значение функции оценки $f(n)$, и именно оно обрабатывается в первую очередь.

Такой подход позволяет существенно сократить число рассматриваемых состояний, направляя поиск в сторону целевого состояния и избегая полного перебора графа. В частности, порядок обработки вершин определяется не расстоянием от начального состояния, как в BFS, а значениями эвристической функции, что обычно реализуется с помощью приоритетной очереди.

Алгоритмы best-first search образуют широкий класс методов, включающий, в частности, жадный поиск (greedy best-first search) и алгоритм A^* [21], различающиеся выбором функции оценки $f(n)$.

Greedy best-first search

Greedy best-first search (GBFS, жадный поиск «лучший — первый») является вариантом best-first search, в котором функция оценки зависит только от эвристики $h(n)$, оценивающей расстояние от текущего состояния до целевого. На каждой итерации алгоритм выбирает для обработки вершину с минимальным значением $h(n)$, игнорируя стоимость пути от начального состояния. Такой подход позволяет быстро направлять поиск в сторону цели, однако не гарантирует нахождение кратчайшего пути и может приводить к неоптимальным решениям.

Algorithm 3 Greedy Best-First Search (GBFS)

Require: граф $G = (V, E)$, начальная вершина s , целевая вершина t , эвристика $h(\cdot)$

Ensure: путь от s до t (если существует)

```

1: создать приоритетную очередь  $Q$ 
2:  $Q \leftarrow \{s\}$ 
3:  $parent[s] \leftarrow nil$ 
4: отметить  $s$  как посещённую
5: while  $Q \neq \emptyset$  do
6:    $v \leftarrow$  извлечь вершину с минимальным  $h(v)$  из  $Q$ 
7:   if  $v = t$  then
8:     return восстановленный путь по  $parent$ 
9:   end if
10:  for all  $u \in Adj(v)$  do
11:    if  $u$  не посещена then
12:      отметить  $u$  как посещённую
13:       $parent[u] \leftarrow v$ 
14:      добавить  $u$  в  $Q$  с приоритетом  $h(u)$ 
15:    end if
16:  end for
17: end while
18: return "пути не существует"
```

Несмотря на эффективность по сравнению с неэвристическими методами, greedy best-first search не гарантирует нахождение оптимального решения и сильно зависит от качества эвристической функции. В случае с графами Кэли большого размера, таких как пространство состояний кубика Рубика, данный метод может существенно отклоняться от целевого состояния при недостаточно информативной эвристике, поскольку поиск направляется исключительно локальными оценками $h(n)$, не учитывающими стоимость пройденного пути.

A* search

Алгоритм A* является расширением greedy best-first search и использует функцию оценки вида $f(n) = g(n) + h(n)$, где $g(n)$ — стоимость пути от начального состояния до текущего узла, а $h(n)$ — эвристическая оценка расстояния до цели. В отличие от жадного поиска, A* учитывает как уже пройденную стоимость пути, так и предполагаемую оставшуюся стоимость, что при допустимой эвристике обеспечивает нахождение оптимального решения. A* является одним из наиболее широко используемых алгоритмов эвристического поиска благодаря балансу между эффективностью и оптимальностью.

Для алгоритма A* важным свойством эвристической функции является её допустимость. Эвристика $h(n)$ называется допустимой, если она не переоценивает истинную стоимость пути до цели $h^*(n)$, то есть $h(n) \leq h^*(n)$. Такое свойство обеспечивает корректность A*, поскольку эвристика всегда даёт нижнюю оценку оставшегося расстояния и не исключает оптимальные решения [21].

Алгоритм A* при использовании допустимой эвристики гарантирует нахождение оптимального решения, однако его эффективность также существенно зависит от качества эвристической функции. Для графов Кэли большого размера применение A* на практике затруднено, поскольку при слабой эвристике алгоритм деградирует до полного перебора пространства состояний. В таких задачах, включая кубик Рубика большого размера, построение достаточно информативной эвристики является сложной самостоятельной

Algorithm 4 A* search

Require: граф $G = (V, E)$, начальная вершина s , целевая вершина t , эвристика $h(\cdot)$

Ensure: кратчайший путь от s до t

```
1: создать приоритетную очередь  $Q$ 
2:  $g[s] \leftarrow 0$ 
3:  $f[s] \leftarrow h(s)$ 
4:  $parent[s] \leftarrow \text{nil}$ 
5: добавить  $s$  в  $Q$  с приоритетом  $f(s)$ 
6: while  $Q \neq \emptyset$  do
7:    $v \leftarrow$  извлечь вершину с минимальным  $f(v)$ 
8:   if  $v = t$  then
9:     return путь, восстановленный по  $parent$ 
10:  end if
11:  for all  $u \in Adj(v)$  do
12:     $tentative \leftarrow g[v] + w(v, u)$ 
13:    if  $u$  не посещена or  $tentative < g[u]$  then
14:       $g[u] \leftarrow tentative$ 
15:       $f[u] \leftarrow g[u] + h(u)$ 
16:       $parent[u] \leftarrow v$ 
17:      добавить  $u$  в  $Q$  с приоритетом  $f(u)$ 
18:    end if
19:  end for
20: end while
21: return "пути не существует"
```

задачей, без которой A* теряет свои вычислительные преимущества и становится непригодным из-за экспоненциального роста числа рассматриваемых состояний.

Лучевой поиск

Лучевой поиск (beam search) [23] является одним из наиболее распространённых алгоритмов эвристического поиска в задачах с экспоненциальным ростом пространства состояний. Данный метод представляет собой модификацию best-first search, в которой на каждом шаге поиска сохраняется фиксированное число наиболее перспективных состояний, определяемых значением эвристической функции $h(n)$. Остальные состояния отбрасываются.

ся, что позволяет существенно ограничить объём вычислений при сохранении фокуса на наиболее вероятных траекториях.

Beam search реализуется как поуровневый эвристический поиск в пространстве состояний с ограничением ширины рассматриваемого множества. Алгоритм начинает с начального состояния и на каждой итерации формирует множество всех его непосредственных преемников. Далее все полученные состояния оцениваются с помощью функции $h(n)$, после чего сохраняются только k состояний с наилучшими значениями данной функции.

Остальные состояния отбрасываются и не участвуют в дальнейших шагах поиска. Процесс повторяется до тех пор, пока не будет достигнуто целевое состояние либо не будут исчерпаны все допустимые вершины.

Algorithm 5 Beam Search (лучевой поиск)

Require: граф $G = (V, E)$, начальная вершина s , целевая вершина t , эвристика $h(n)$, ширина луча k

Ensure: путь от s до t (если существует)

```
1:  $B_0 \leftarrow \{s\}$ 
2:  $parent[s] \leftarrow nil$ 
3:  $i \leftarrow 0$ 
4: while  $B_i \neq \emptyset$  do
5:    $C \leftarrow \emptyset$ 
6:   for all  $v \in B_i$  do
7:     for all  $u \in Adj(v)$  do
8:       добавить  $u$  в  $C$ 
9:        $parent[u] \leftarrow v$ 
10:    end for
11:  end for
12:  упорядочить элементы  $C$  по возрастанию  $h(u)$ 
13:   $B_{i+1} \leftarrow$  первые  $k$  элементов из  $C$ 
14:  if  $t \in B_{i+1}$  then
15:    return восстановленный путь по  $parent$ 
16:  end if
17:   $i \leftarrow i + 1$ 
18: end while
19: return "решение не найдено"
```

Таким образом, beam search осуществляет управляемое сокращение пространства поиска, ограничивая число одновременно рассматриваемых ги-

потенз фиксированной шириной k , что позволяет применять алгоритм в задачах с экспоненциальным ростом числа состояний, включая графы Кэли, возникающие в задачах поиска решений кубика Рубика. Эффективность метода определяется выбором ширины луча и качеством используемой эвристической функции, а его теоретические свойства и успешность применения в задачах, связанных с машинным обучением, подробно обосновываются в современных исследованиях [30].

3.3. Архитектуры нейросетей для предсказания эвристик

Эффективность алгоритмов эвристического поиска, таких как A^* и beam search, в решающей степени определяется качеством используемой функции оценки состояния. Классические подходы к построению эвристик, включая pattern databases и различные аналитические оценки, требуют существенных вычислительных затрат на этапе предварительной обработки и часто плохо масштабируются на задачи большой размерности.

В связи с этим в последние годы активно развивается направление, связанное с обучением функций оценки с использованием нейросетевых моделей. В данном подходе эвристическая функция $h(n)$ рассматривается как неизвестная функция расстояния до целевого состояния и аппроксимируется параметризованной моделью $h_\theta(n)$, обучаемой на примерах состояний и соответствующих им расстояний или их оценок.

Использование нейросетевых функций оценки позволяет отказаться от явного хранения больших таблиц и адаптировать эвристику к структуре конкретного пространства состояний. При этом такие методы сохраняют совместимость с классическими алгоритмами поиска: обученная функция может напрямую использоваться в A^* , beam search и их модификациях, направляя процесс поиска к целевому состоянию.

Данный подход лежит в основе современных методов решения задач поиска в графах Кэли, включая задачи, связанные с кубиком Рубика, где пространство состояний обладает сложной структурой и экспоненциальным ростом.

Многослойный перцептрон

В качестве базовой архитектуры для задания нейросетевой функции оценки широко используется многослойный перцептрон (Multi-Layer Perceptron, MLP) [24]. Данная архитектура представляет собой полносвязную нейронную сеть прямого распространения, состоящую из последовательности слоёв, каждый из которых реализует аффинное преобразование входа с последующим применением нелинейной функции активации (рис. 6).

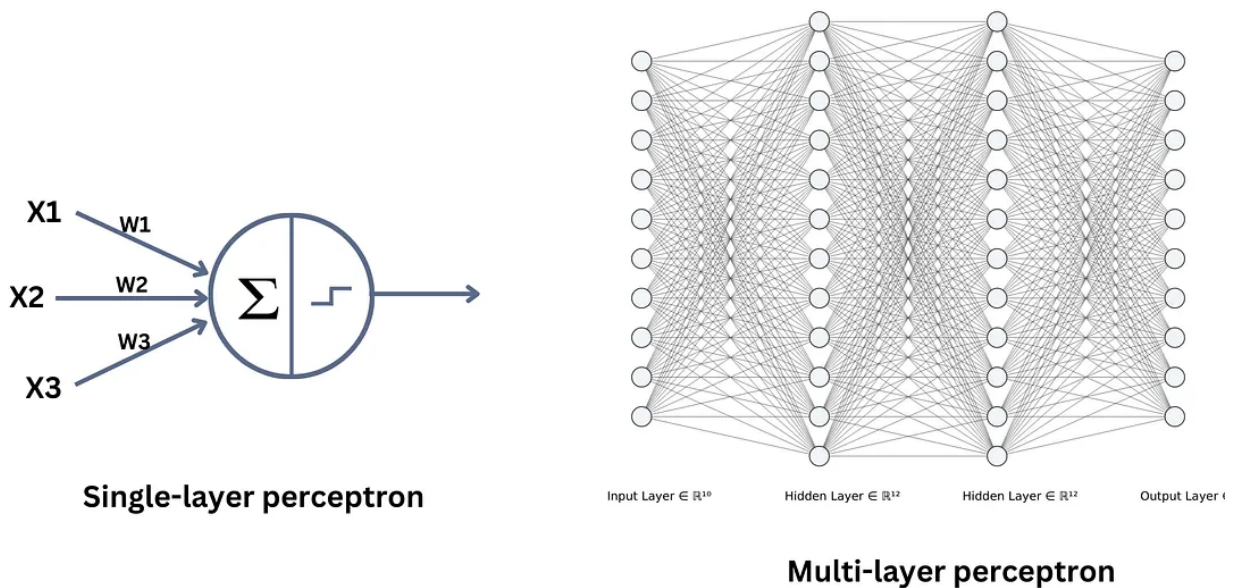


Рис. 6: Схема работы многослойного перцептрона (MLP) [25]

Формально, MLP задаёт параметризованную функцию $h_{\theta}(n)$, отображающую описание состояния в скалярную оценку. Пусть вход x_0 соответствует векторному представлению состояния. Тогда для слоя i вычисление имеет вид

$$x_i = \sigma_i(W_i x_{i-1} + b_i),$$

где W_i и b_i — параметры слоя, а σ_i — нелинейная функция активации, применяемая поэлементно. Итоговое значение $h_{\theta}(n)$ формируется на выходном слое сети.

Наличие нелинейных функций активации позволяет сети аппроксимировать сложные зависимости между состоянием и расстоянием до цели, поскольку композиция линейных преобразований без нелинейностей сводится

к одному линейному отображению. Благодаря этому MLP может использоваться как универсальный аппроксиматор функции оценки, что делает его удобным инструментом для построения эвристик в задачах поиска на графах большого размера.

В контексте задач поиска на графах Кэли входом сети является кодировка состояния, а выходом — оценка расстояния до целевой вершины. Полученная функция $h_\theta(n)$ далее используется в алгоритмах поиска, таких как beam search, для ранжирования состояний и направления процесса поиска.

В практических реализациях архитектура MLP часто дополняется вспомогательными компонентами, улучшающими свойства обучения. В частности, между слоями могут добавляться операции нормализации (например, batch normalization или layer normalization), позволяющие стабилизировать распределение активаций и повысить обобщающую способность модели.

Кроме того, для облегчения обучения глубоких сетей используются остаточные соединения (residual connections), при которых вход слоя частично добавляется к его выходу. Такой механизм способствует лучшей проводимости градиентов при обратном распространении ошибки и позволяет эффективно обучать более глубокие архитектуры.

Графовая нейросеть

В задачах, где входные данные имеют графовую структуру, более естественным выбором архитектуры являются графовые нейронные сети (Graph Neural Networks, GNN) [26]. В отличие от многослойного перцептрона, который работает с фиксированными векторами признаков, GNN непосредственно оперируют графами, учитывая как свойства вершин, так и структуру связей между ними.

Основой большинства современных GNN является механизм распространения сообщений (message passing), при котором представления вершин итеративно обновляются за счёт агрегации информации от соседних вершин (рис. 7).

Пусть задан граф $G = (V, E)$, где каждой вершине $v \in V$ сопоставлен

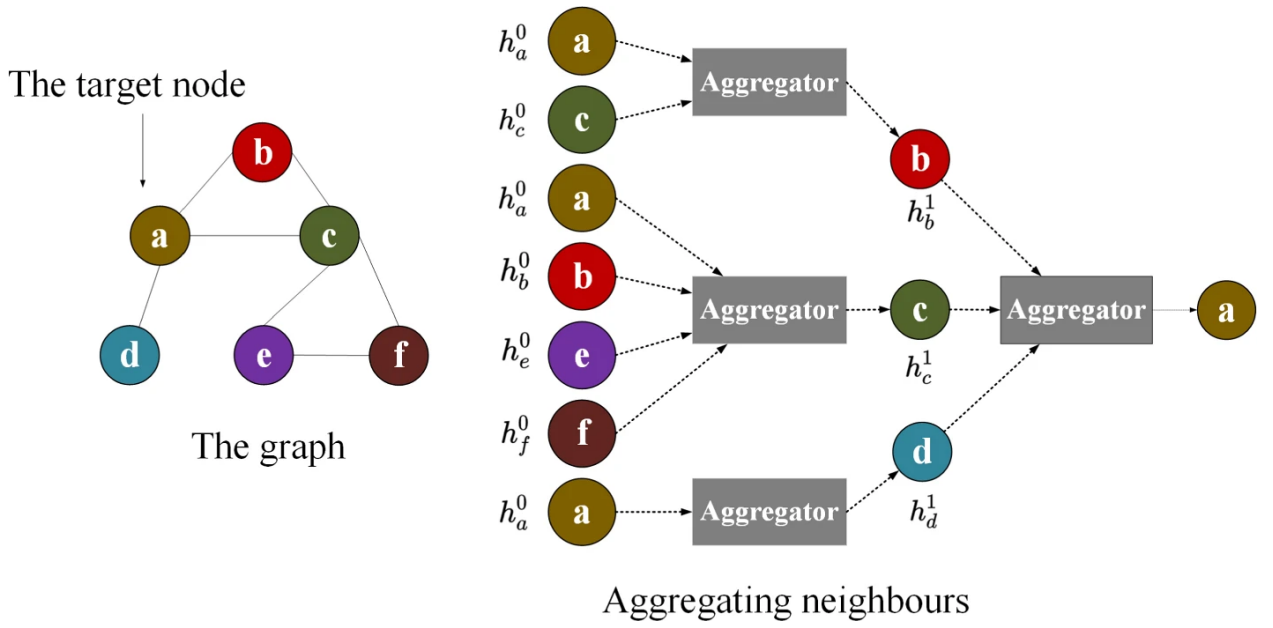


Рис. 7: Схема агрегации признаков соседних узлов в графовой нейронной сети (GNN) [27]

вектор признаков $h_v^{(0)}$. Тогда на l -м слое обновление представления вершины имеет вид:

$$h_v^{(l+1)} = \phi^{(l)} \left(h_v^{(l)}, \bigoplus_{u \in Adj(v)} \psi^{(l)}(h_v^{(l)}, h_u^{(l)}) \right),$$

где ψ — функция формирования сообщения, $\bigoplus$ — операция агрегации (например, сумма или среднее), а ϕ — функция обновления.

Таким образом, на каждом слое вершина агрегирует информацию из своего локального окружения, а при увеличении числа слоёв учитываются всё более дальние вершины графа. Итоговое представление может использоваться как для оценки отдельных вершин, так и для получения глобальной характеристики всего графа.

В контексте задач поиска на графах Кэли вершины графа соответствуют состояниям, а рёбра — допустимым переходам. GNN позволяет учитывать структуру локального окружения состояния, агрегируя информацию о соседних состояниях, что потенциально даёт более информативную функцию оценки по сравнению с моделями, не учитывающими структуру графа.

Графовые нейронные сети обладают рядом существенных преимуществ: они естественным образом работают с графовыми структурами данных, позволяя учитывать как признаки вершин, так и связи между ними, эффективно

извлекают информацию о взаимозависимостях элементов и применимы к графам произвольной структуры и размера, поддерживая различные типы задач, включая классификацию вершин, предсказание рёбер и анализ графов в целом. При этом такие модели способны обобщать на новые графы и использовать контекст соседних вершин, что делает их особенно полезными в задачах с выраженной структурой зависимостей. Вместе с тем, GNN имеют и ряд ограничений: они требуют значительных вычислительных ресурсов из-за необходимости многократной агрегации информации по рёбрам, плохо масштабируются на очень большие графы без специальных техник, чувствительны к качеству заданной структуры графа, а также испытывают трудности с учётом дальних зависимостей и могут терять различимость представлений при увеличении глубины сети (эффект over-smoothing).

В задачах поиска на графах Кэли применение GNN ограничено необходимостью работы с локальной структурой графа, что может быть затруднено при очень большом числе вершин. В связи с этим на практике часто используются более простые архитектуры, такие как MLP, работающие с локальным представлением состояния без явного обращения к структуре графа.

Механизм самовнимания

В качестве альтернативного подхода к построению функций оценки, учитывающих взаимосвязи между элементами входа, широко используется механизм самовнимания (self-attention) [28], лежащий в основе архитектуры Transformer. Данный механизм позволяет каждой компоненте входного представления учитывать информацию обо всех остальных компонентах, формируя контекстно-зависимое представление состояния.

Пусть входное состояние представлено вектором или последовательностью векторов $X = (x_1, x_2, \dots, x_n)$, где каждый элемент кодирует часть состояния. В self-attention каждый такой вектор преобразуется в три различных представления:

$$q_i = W^Q x_i, k_i = W^K x_i, v_i = W^V x_i,$$

где W^Q, W^K, W^V — обучаемые матрицы. Таким образом, исходное представ-

ление состояния проецируется в три различных пространства признаков.

Полученные представления имеют различный функциональный смысл. Вектор q_i можно интерпретировать как запрос, определяющий, какую информацию текущий элемент хочет получить от остальных. Вектор k_i задаёт описание того, какую информацию данный элемент может предоставить другим, а вектор v_i содержит собственно передаваемое содержимое.

Таким образом, каждый элемент входа одновременно выступает в двух ролях: как источник информации (через k_i, v_i) и как её потребитель (через q_i). Это позволяет модели учитывать зависимости между всеми частями входного состояния и формировать представления, зависящие от глобального контекста.

На следующем этапе для каждого элемента входа вычисляется взвешенная комбинация значений v_j других элементов. Веса этой комбинации определяются степенью соответствия между запросом q_i и ключами k_j , что формально записывается как

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

где матрица QK^T отражает попарное сходство элементов, а нормировка через функцию softmax обеспечивает корректное масштабирование.

Таким образом, выход self-attention для каждого элемента представляет собой агрегированное представление, в котором учитывается вклад всех остальных элементов входа, взвешенный их релевантностью (рис. 8). Это позволяет формировать контекстно-зависимые представления, отражающие глобальную структуру входного состояния.

К основным преимуществам self-attention относится способность эффективно учитывать дальние зависимости между элементами входа и формировать контекстно-зависимые представления без необходимости задания явной структуры графа, а также высокая гибкость и возможность параллельной обработки входных данных. В то же время данный механизм обладает квадратичной сложностью по длине входа, что ограничивает его применение в задачах с очень большим числом элементов, а также требует аккуратного

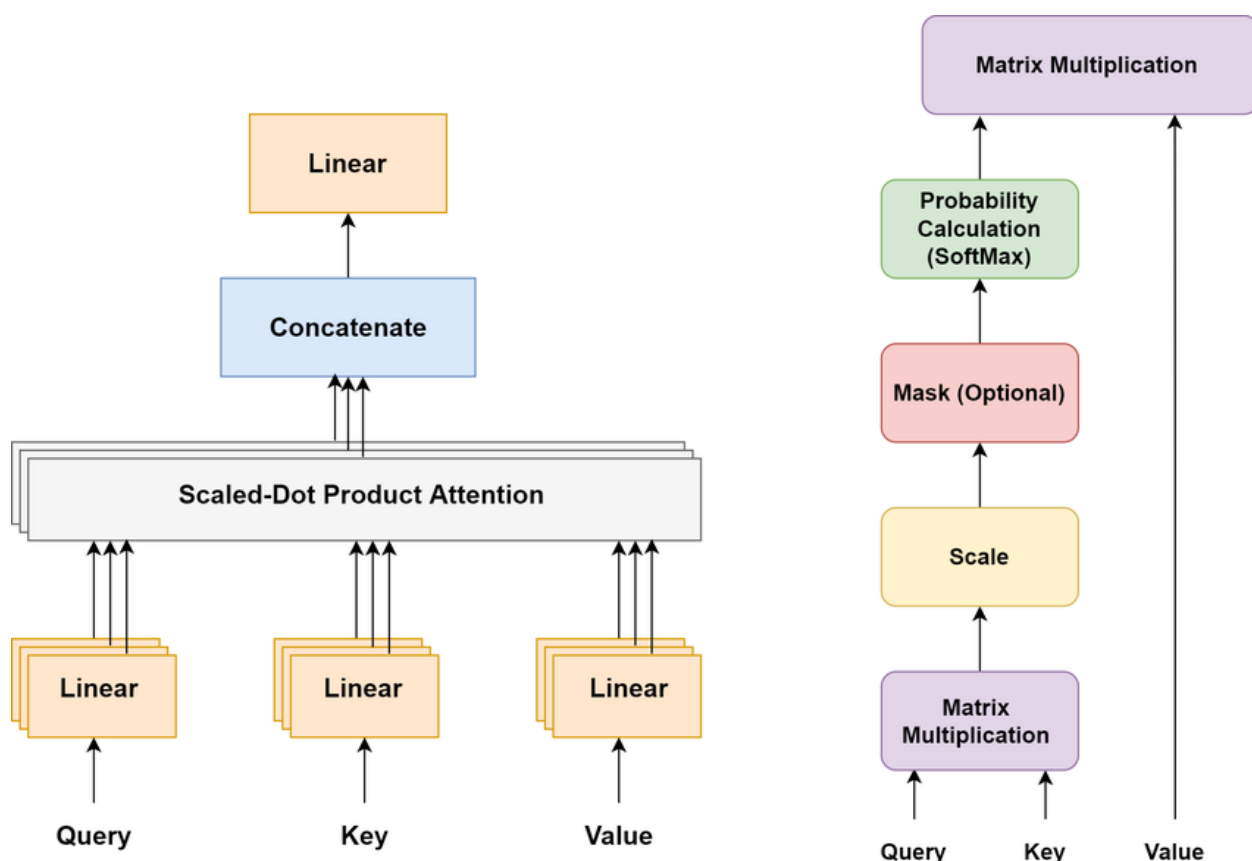


Рис. 8: Схема работы механизма self-attention [28]

выбора способа кодирования входного состояния.

В задачах поиска на графах Кэли self-attention может использоваться для построения функций оценки, учитывающих взаимодействие различных компонент состояния, однако его применение требует компактного и информативного представления состояния, поскольку вычислительная сложность механизма быстро возрастает с увеличением размерности входа.

3.4. Алгоритмы генерации тренировочной выборки

Качество обучаемой функции оценки напрямую определяется характеристиками используемой тренировочной выборки. В задачах обучения нейросетевых эвристик требуется большое количество размеченных состояний с известными или оценёнными расстояниями до целевого состояния, что делает построение такой выборки одной из ключевых задач. При этом получение обучающих данных может быть существенно более сложным и ресурсоёмким, чем последующее обучение модели.

В общем случае тренировочная выборка представляет собой набор примеров, используемых для подбора параметров модели, и именно её качество во многом определяет точность и обобщающую способность итоговой функции оценки.

В задачах поиска на графах Кэли формирование обучающих данных осложняется огромным размером пространства состояний и отсутствием явных разметок. В связи с этим используются специализированные алгоритмы генерации данных, позволяющие получать пары вида «состояние — оценка расстояния» без полного перебора графа. Такие методы включают случайные блуждания, синтетическую генерацию данных и различные процедуры приближённой оценки расстояний, позволяющие формировать обучающую выборку с контролируемыми вычислительными затратами.

В данном разделе рассматриваются основные подходы к генерации тренировочной выборки, применяемые в задачах обучения функций оценки для поиска в графах большого размера.

Случайные блуждания

Одним из наиболее простых и широко используемых подходов к генерации обучающих данных в задачах поиска на графах является метод случайных блужданий (random walks). Данный метод основан на стохастическом обходе графа: начиная с некоторой вершины, на каждом шаге выполняется переход в одну из соседних вершин, выбранную случайным образом.

Формально, случайное блуждание задаёт последовательность состояний $(s_0, s_1, \dots, s_T)$, где s_0 — начальная вершина, а каждое следующее состояние s_{t+1} выбирается случайно из множества соседей вершины s_t . Такой процесс обладает марковским свойством: выбор следующего состояния зависит только от текущего и не зависит от предыдущей траектории.

В контексте обучения функций оценки случайные блуждания используются для генерации пар вида «состояние — расстояние до цели». Для этого блуждание часто запускается из целевого состояния: на каждом шаге выполняется случайный переход, а для посещённых состояний фиксируется длина пройденного пути. Таким образом, для состояния s_t естественной оценкой

расстояния является число шагов t , затраченное на достижение этого состояния из целевой вершины.

Повторяя данный процесс множество раз, можно получить большой набор разнообразных состояний, распределённых по пространству состояний, вместе с соответствующими оценками расстояний. Такой способ генерации не требует полного перебора графа и позволяет контролировать распределение обучающих примеров за счёт длины блужданий и их количества.

Algorithm 6 Генерация обучающей выборки с помощью случайных блужданий

Require: граф $G = (V, E)$, целевая вершина s^* , число блужданий N , максимальная длина T

Ensure: набор пар (s, d)

```

1:  $D \leftarrow \emptyset$ 
2: for  $i = 1 \dots N$  do
3:    $s \leftarrow s^*$ 
4:   for  $t = 1 \dots T$  do
5:     выбрать случайного соседа  $u \in Adj(s)$ 
6:      $s \leftarrow u$ 
7:     добавить  $(s, t)$  в  $D$ 
8:   end for
9: end for
10: return  $D$ 

```

Случайные блуждания можно рассматривать как способ стохастического “зондирования” пространства состояний. В отличие от систематических методов перебора, они позволяют эффективно получать выборку состояний даже в графах очень большого размера, при этом естественным образом охватывая различные области пространства. Кроме того, последовательности состояний, получаемые в ходе блужданий, могут интерпретироваться как траектории в графе, что делает их удобными для обучения моделей, способных учитывать динамику переходов между состояниями.

К преимуществам метода относится простота реализации, отсутствие необходимости в полном знании графа и возможность генерации практически неограниченного объёма обучающих данных. Вместе с тем, случайные блуждания могут приводить к неравномерному покрытию пространства состояний, поскольку вероятность посещения вершин зависит от структуры

графа, а также давать шумные оценки расстояний, не совпадающие с истинными кратчайшими путями.

В задачах поиска на графах Кэли, включая пространство состояний кубика Рубика, случайные блуждания представляют собой эффективный способ генерации обучающих данных без полного перебора состояний. Это делает их одним из ключевых инструментов при обучении нейросетевых функций оценки в условиях экспоненциального роста пространства состояний.

Случайные блуждания без возвратов

Важной модификацией метода случайных блужданий являются блуждания без возвратов (*non-backtracking random walks*), в которых на каждом шаге запрещён переход обратно в вершину, посещённую на предыдущем шаге. Иными словами, если на шаге t был выполнен переход $s_{t-1} \rightarrow s_t$, то на следующем шаге запрещается переход $s_t \rightarrow s_{t-1}$.

Такая модификация нарушает марковское свойство стандартного случайного блуждания (поскольку теперь требуется помнить предыдущее состояние), однако позволяет избежать тривиальных возвратов и делает траектории более «развёрнутыми» в пространстве состояний.

В контексте генерации обучающих данных это приводит к более эффективному покрытию пространства состояний: последовательные шаги с меньшей вероятностью компенсируют друг друга, что уменьшает долю коротких циклов и повышает разнообразие получаемых состояний. При этом, как и в стандартном случае, длина пройденного пути может использоваться в качестве оценки расстояния до целевого состояния.

При реализации данного подхода необходимо учитывать, что в некоторых состояниях множество допустимых переходов может оказаться пустым (например, если вершина имеет степень 1). В этом случае обычно допускается возврат к предыдущему состоянию или выполняется перезапуск блуждания. Однако для графов Кэли с числом генераторов, большим единицы, включая графы Кэли кубика Рубика, такая ситуация не возникает, поскольку из любой вершины существует как минимум два различных допустимых перехода.

В отличие от стандартного случайного блуждания, данный вариант ис-

Algorithm 7 Генерация выборки с помощью случайных блужданий без возвратов

Require: граф $G = (V, E)$, целевая вершина s^* , число блужданий N , максимальная длина T

Ensure: набор пар (s, d)

```
1:  $D \leftarrow \emptyset$ 
2: for  $i = 1 \dots N$  do
3:    $s \leftarrow s^*$ 
4:    $prev \leftarrow \text{nil}$ 
5:   for  $t = 1 \dots T$  do
6:      $Candidates \leftarrow Adj(s)$ 
7:     if  $prev \neq \text{nil}$  then
8:       удалить  $prev$  из  $Candidates$ 
9:     end if
10:    выбрать случайного  $u \in Candidates$ 
11:     $prev \leftarrow s$ 
12:     $s \leftarrow u$ 
13:    добавить  $(s, t)$  в  $D$ 
14:   end for
15: end for
16: return  $D$ 
```

ключает немедленные возвраты, что приводит к более быстрому удалению от начального состояния и снижает коррелированность последовательных состояний.

Случайные блуждания без повторного посещения вершин

Более строгой модификацией случайных блужданий являются блуждания без повторного посещения вершин. Пусть $Visited$ — множество всех ранее посещённых состояний во всех сгенерированных траекториях. Тогда на каждом шаге допустимые переходы определяются множеством $Adj(s) \setminus Visited$.

В данном подходе запрещается повторное посещение любых ранее встреченных состояний, независимо от траектории. Таким образом, формируемые последовательности состояний $\{(s_0^{(i)}, s_1^{(i)}, \dots, s_T^{(i)})\}$ представляют собой простые пути в графе, которые могут пересекаться только на начальных

Algorithm 8 Генерация выборки с помощью случайных блужданий без повторного посещения вершин

Require: граф $G = (V, E)$, целевая вершина s^* , число блужданий N , максимальная длина T

Ensure: набор пар (s, d)

```
1:  $D \leftarrow \emptyset$ 
2:  $Visited \leftarrow \{s^*\}$ 
3: for  $i = 1 \dots N$  do
4:    $current[i] \leftarrow s^*$ 
5: end for
6: for  $t = 1 \dots T$  do
7:    $VisitedNow \leftarrow \emptyset$ 
8:   for  $i = 1 \dots N$  do
9:      $s \leftarrow current[i]$ 
10:     $Candidates \leftarrow Adj(s) \setminus Visited$ 
11:    if  $Candidates = \emptyset$  then
12:      continue
13:    end if
14:    выбрать случайного  $u \in Candidates$ 
15:     $current[i] \leftarrow u$ 
16:    добавить  $u$  в  $VisitedNow$ 
17:    добавить  $(u, t)$  в  $D$ 
18:  end for
19:   $Visited \leftarrow Visited \cup VisitedNow$ 
20: end for
21: return  $D$ 
```

префиксах одинаковой длины.

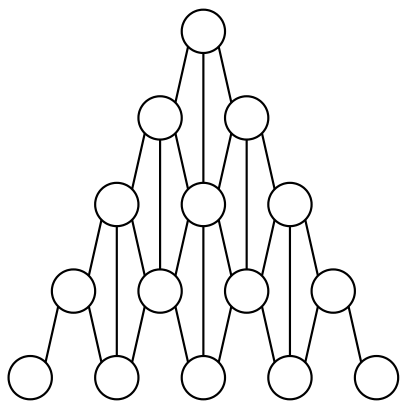
Такой подход приводит к построению набора пар «состояние — оценка расстояния», в котором каждому состоянию соответствует единственная оценка. Отсутствие повторных посещений исключает образование циклов и снижает вероятность локального блуждания в уже исследованных областях.

Однако введение глобального множества запрещённых вершин приводит к быстрому исчерпанию пространства состояний: по мере роста $Visited$ число допустимых переходов уменьшается, что ограничивает длину траекторий и общее число сгенерированных примеров. В предельном случае процесс может завершиться преждевременно, если для текущего состояния не остаётся допустимых переходов.

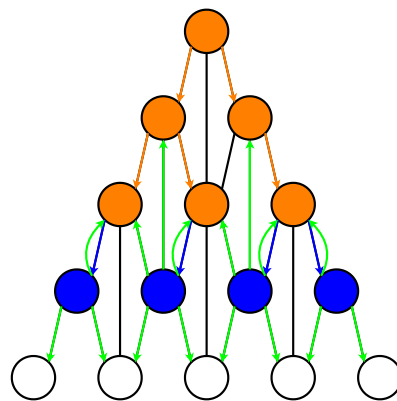
Таким образом, данный подход эффективно устраняет цикличность и повторные посещения состояний, обеспечивая генерацию простых путей. Однако использование глобального множества запрещённых вершин приводит к ограничению исследуемого подграфа и возможному сокращению длины траекторий, что потенциально снижает разнообразие выборки и покрытие графа ей.

Сравнение трёх рассмотренных подходов к генерации обучающей выборки проиллюстрировано на рис. 9. Синим выделены состояния достигнутые на последнем шаге блужданий и последние совершённые переходы, оранжевым — остальные состояния, достигнутые во время блужданий и прочие совершённые переходы, зелёным — все разрешённые переходы из текущих состояний, красным — все запрещённые переходы.

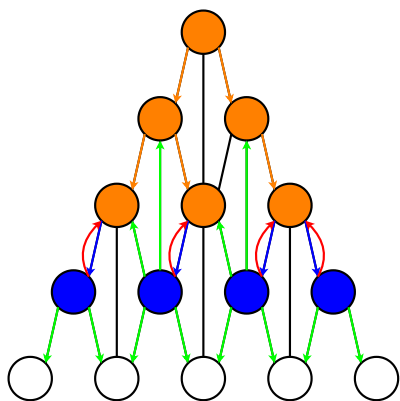
На рис. 9а показан исходный граф состояний, используемый как базовая структура для демонстрации различных стратегий генерации траекторий. На рис. 9b представлены стандартные случайные блуждания, в которых допускаются повторные посещения вершин и немедленные возвраты, вследствие чего траектории содержат циклы и локальные колебания. На рис. 9с изображены случайные блуждания без немедленных возвратов: алгоритм запрещает переход обратно в вершину предыдущего шага, благодаря чему траектории становятся более направленными и быстрее удаляются от исходного состояния. На рис. 9d показаны случайные блуждания без повторного посещения вершин, где каждая вершина посещается не более одного раза. В результате формируются простые пути без циклов, обеспечивающие более равномерное покрытие пространства состояний.



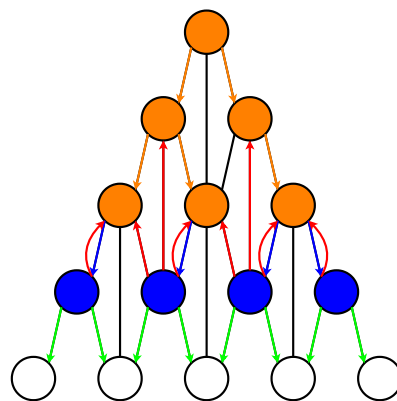
(a) Исходный граф.



(b) Стандартные случайные блуждания.



(c) Блуждания без немедленных возвратов.



(d) Блуждания без повторного посещения вершин.

Рис. 9: Визуализация трёх методов генерации обучающей выборки.

4. Эксперименты

В экспериментах используется алгоритм лучевого поиска (beam search) как метод, позволяющий одновременно рассматривать множество альтернативных направлений поиска. На каждом шаге алгоритм поддерживает фиксированное число состояний (луч), отбираемых по значению эвристической функции, что обеспечивает баланс между полнотой перебора и вычислительной сложностью.

Ключевым гиперпараметром является ширина луча W : её увеличение улучшает качество решений за счёт более широкого покрытия пространства поиска, однако сопровождается ростом вычислительных затрат и потребления памяти.

Основная идея экспериментов — исследование зависимости качества решения от W , в том числе возможности повышения доли успешно решённых задач при уменьшении ширины луча.

Вычислительная конфигурация: одна GPU NVIDIA A100 PCIe 40 ГБ и CPU AMD EPYC 7742 (64 ядра). Все эксперименты проводились на графах Кэли для кубиков Рубика размеров $3 \times 3 \times 3$, $4 \times 4 \times 4$, $5 \times 5 \times 5$ в метрике четвертьходов (QTM) с множествами генераторов мощностью 18, 24 и 30 соответственно. Состояние кубика представляется вектором цветов фасцетов длиной 54, 96 и 150 элементов. В качестве тестовых данных использовались выборки из соревнования Kaggle Santa 2023: 82, 43 и 19 начальных состояний для трёх размеров кубика соответственно.

4.1. Эксперименты с обучающим датасетом

В данном разделе исследуется влияние способа формирования обучающей выборки на качество работы алгоритма поиска. Сравниваются три подхода к генерации пар «состояние — оценка расстояния». Архитектура нейросети (MLP) и алгоритм поиска во всех вариантах фиксированы.

Описание экспериментов

Рассматриваются следующие варианты генерации обучающих данных:

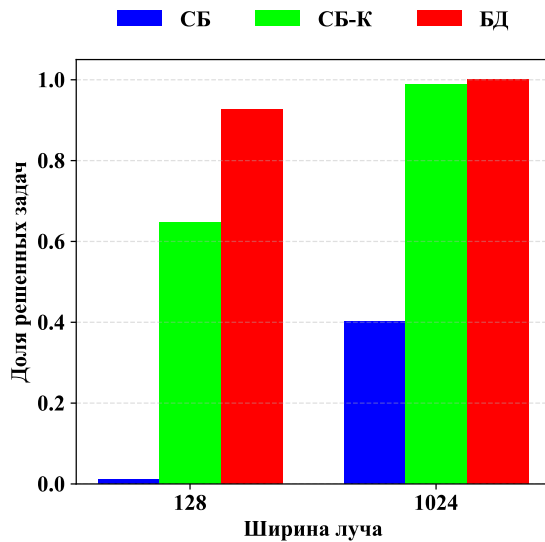
СБ — случайные блуждания (базовый вариант). На каждой эпохе генерируются новые траектории глубиной $T = 26, 45, 65$ ходов для кубиков $3 \times 3 \times 3$, $4 \times 4 \times 4$, $5 \times 5 \times 5$ соответственно, без ограничений на повторные посещения вершин. Число траекторий на эпоху равно $\lfloor 10^6/T \rfloor$; обучение проводится в течение 8192 эпох. Подход обеспечивает высокое разнообразие примеров за счёт динамической генерации, однако одно и то же состояние может встречаться с различными метками расстояния, порождая противоречия в разметке.

СБ-К — случайные блуждания с консистентными метками. Параметры генерации идентичны базовому варианту. Введено единственное изменение: на каждой эпохе поддерживается глобальное множество *Visited*, и переходы в уже посещённые вершины запрещены. Это устраняет неоднозначность разметки внутри эпохи при сохранении динамической генерации.

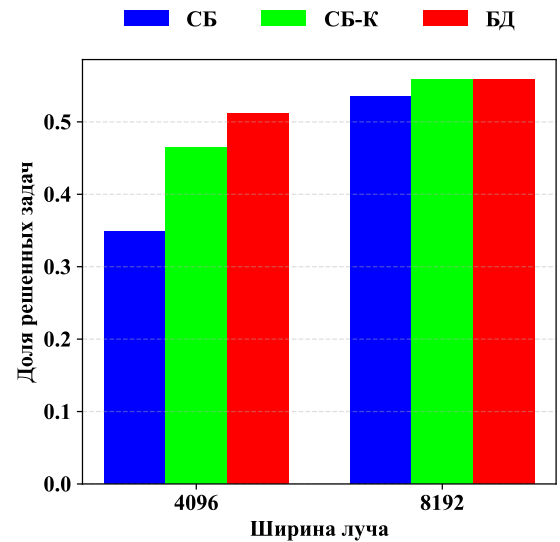
БД — один большой датасет с консистентными метками. Единая фиксированная выборка размером $\lfloor 128 \cdot 10^6/T \rfloor$ состояний генерируется заранее с глобальным запретом повторных посещений. Обучение проводится в течение 64 эпох на фиксированном распределении. Вариант устраняет неоднозначность разметки на всём протяжении обучения, но снижает разнообразие примеров по сравнению с динамической генерацией.

Результаты

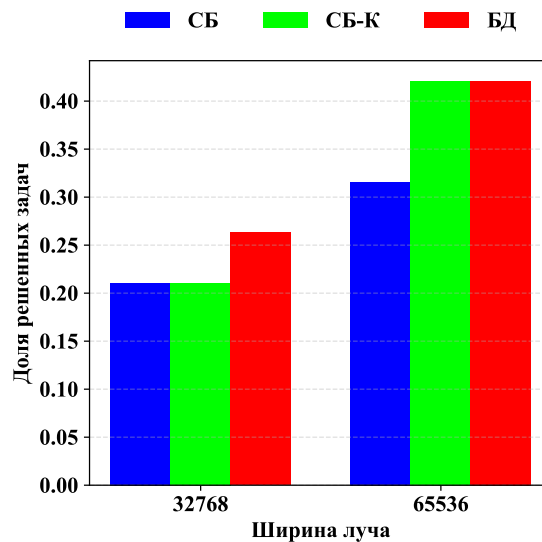
На рисунке 10а показана зависимость доли решённых задач от ширины луча W для кубика $3 \times 3 \times 3$. Результаты для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$ представлены на рисунках 10b и 10c соответственно. На всех графиках по горизонтальной оси отложена ширина луча W , а по вертикальной оси — доля успешно решённых задач. Различными цветами обозначены используемые методы формирования обучающей выборки: базовый вариант случайных блужданий (**СБ**) соответствует синим столбцам, случайные блуждания без повторного посещения вершин (**СБ-К**) — зелёным столбцам, а один большой датасет с консистентными метками (**БД**) — красным столбцам.



(a) Кубик $3 \times 3 \times 3$ (82 задачи).



(b) Кубик $4 \times 4 \times 4$ (43 задачи).



(c) Кубик $5 \times 5 \times 5$ (19 задач).

Рис. 10: Доля решённых задач в зависимости от ширины луча W и метода генерации обучающего датасета.

Выводы

Анализ результатов показывает, что консистентность меток расстояния является очень важным фактором. Базовый вариант СБ, допускающий противоречивую разметку одного и того же состояния, демонстрирует существенно более низкое качество поиска: для кубика $3 \times 3 \times 3$ (рис. 10а) при ширине луча $W = 128$ доля решённых задач составляет 0.01, тогда как консистентные варианты (СБ-К и БД) достигают 0.65 и 0.93 соответственно. При

увеличении W до 1024 разрыв сохраняется: СБ даёт 0.40, тогда как БД — 1.00. Для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$ (рис. 10b, 10c) влияние консистентности также сохраняется, хотя и выражено слабее из-за меньшей вероятности пересечения траекторий в большем пространстве состояний. Рекомендуемым вариантом для всех трёх размеров является БД.

4.2. Эксперименты с архитектурами нейросетей

В данном разделе исследуется влияние архитектуры нейросетевой функции оценки на качество поиска. Во всех экспериментах используется один и тот же заранее сгенерированный обучающий датасет (БД), что позволяет исключить влияние различий обучающей выборки и сравнивать непосредственно архитектурные особенности моделей.

В качестве базовой архитектуры рассматривается многослойный перцептрон (MLP), аналогичный модели, используемой в работе CayleyPy для аппроксимации расстояния на графах Кэли. Дополнительно исследуются архитектуры на основе графовых нейронных сетей (GNN) и механизма self-attention (SA).

Описание экспериментов

Модель	Параметры	Вход	Особенность
MLP	~4 млн	Вектор цветов фасетов	Нет явной структуры задачи
GNN	~3 млн	Состояние + все соседи	Локальная структура графа Кэли
SA	~1 млн	Последовательность токенов + CLS	Глобальные зависимости

Таблица 2: Сравниваемые архитектуры нейросетевой функции оценки.

MLP. Многослойный перцептрон принимает на вход плоское векторное представление состояния и предсказывает оценку расстояния до целевого состояния. Архитектура не использует явную информацию о структуре графа Кэли, однако отличается высокой вычислительной эффективностью и низким временем инференса, что особенно важно при использовании beam search с большой шириной луча.

GNN (RGCN [29]). Для каждого состояния дополнительно рассматриваются все соседние состояния, получаемые применением допустимых генераторов. На их основе строится звездообразный граф, где центральная вершина соответствует текущему состоянию, а остальные вершины — его соседям. Для агрегации информации используются два слоя реляционных графовых свёрток (RGCN), учитывающих тип применяемого генератора.

SA (Self-Attention). Состояние кодируется как последовательность токенов с embedding-слоем, позиционными эмбедингами и специальным агрегирующим токеном (CLS). Архитектура включает 3 слоя self-attention с размерностью скрытого пространства $d_{\text{model}} = 192$. Механизм внимания позволяет моделировать взаимодействия между всеми элементами состояния одновременно без явного задания структуры графа.

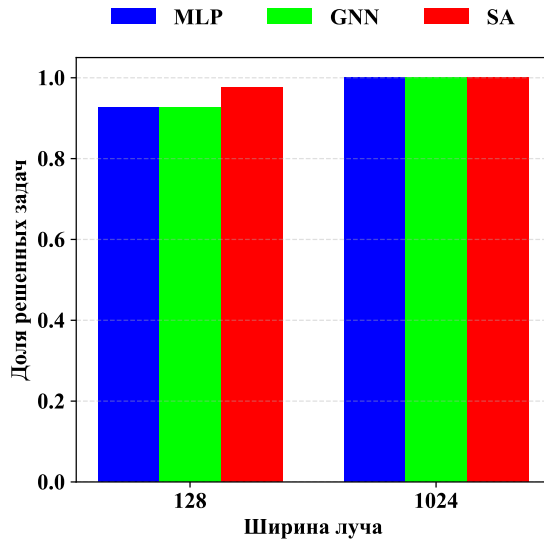
Результаты

Для кубика $3 \times 3 \times 3$ были проведены эксперименты со всеми тремя архитектурами: **MLP**, **GNN** и **SA**. Полученные зависимости доли решённых задач и времени работы от ширины луча представлены на рис. 11a и 12a соответственно.

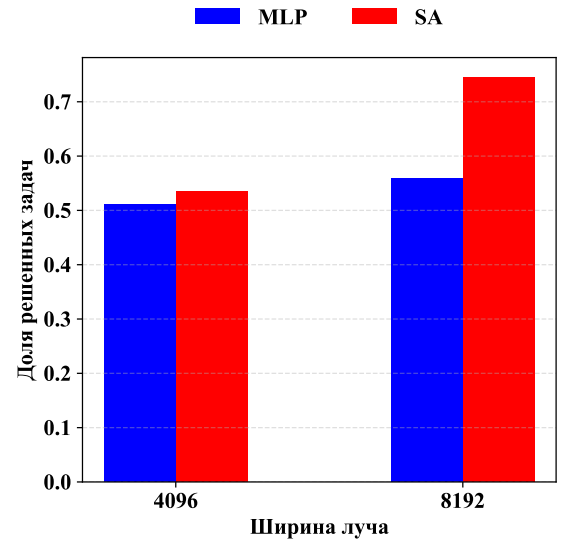
Для кубика $4 \times 4 \times 4$ сравнивались только **MLP** и **SA** (рис. 11b, 12b), поскольку использование **GNN** приводило к слишком большому времени инференса вследствие необходимости обработки соседей каждого состояния в ходе поиска. Для кубика $5 \times 5 \times 5$ дальнейшие эксперименты не проводились, так как self-attention также переставал удовлетворять ограничениям по времени работы beam search, и остался только **MLP**.

Выводы

Результаты экспериментов свидетельствуют о том, что архитектуры GNN и SA дают лишь незначительный прирост качества поиска относительно MLP: для кубика $3 \times 3 \times 3$ (рис. 11a) при $W = 128$ доля решённых задач составляет 0.93 (MLP), 0.93 (GNN) и 0.98 (SA) — разница в пределах нескольких процентных пунктов. При этом время инференса GNN оказывается в 50–60

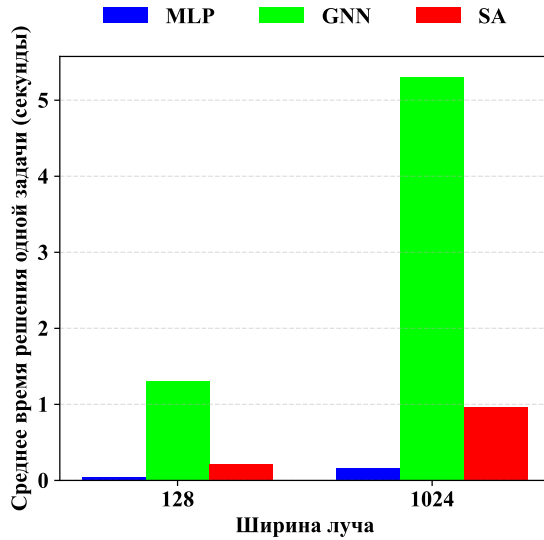


(a) Кубик $3 \times 3 \times 3$ (82 задачи).

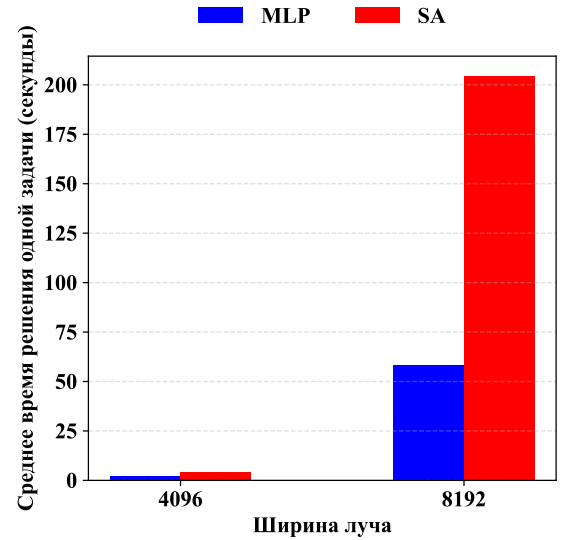


(b) Кубик $4 \times 4 \times 4$ (43 задачи).

Рис. 11: Доля решённых задач в зависимости от ширины луча W и архитектуры нейросети.



(a) Кубик $3 \times 3 \times 3$ (82 задачи).



(b) Кубик $4 \times 4 \times 4$ (43 задачи).

Рис. 12: Время решения одной задачи в зависимости от ширины луча W и архитектуры нейросети.

раз выше, чем у MLP (рис. 12a), что делает его неприменимым для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$ в рамках ограничений по времени. SA при $W = 8192$ на кубике $4 \times 4 \times 4$ требует в среднем около 200 с на задачу против 60 с у MLP (рис. 12b), то есть в 3.3 раза медленнее, что также делает его непрактичным при росте размера кубика. С учётом соотношения качества и скорости инференса MLP был выбран в качестве основной архитектуры.

4.3. Эксперименты с алгоритмом поиска

В данном разделе исследуется влияние модификаций алгоритма поиска на эффективность решения задач. Во всех экспериментах используется одна и та же эвристическая функция — многослойный перцептрон (MLP), обученный на датасете **БД**, что позволяет изолировать влияние именно алгоритмических изменений.

В качестве базового алгоритма используется лучевой поиск (beam search), поскольку он позволяет эффективно контролировать компромисс между качеством поиска и вычислительными затратами с помощью параметра ширины луча W . В отличие от A^* и greedy best-first search, beam search менее чувствителен к локальным ошибкам эвристической функции благодаря одновременному рассмотрению нескольких направлений поиска.

Описание экспериментов

Сравниваются три модификации лучевого поиска:

ЛП — базовый лучевой поиск. Стандартный лучевой поиск без дополнительных ограничений. На каждом шаге сохраняются W лучших состояний по значению эвристической функции. Поиск завершается при нахождении решения либо достижении ограничения на число итераций.

ЛП-БП — лучевой поиск с локальным запретом повторных посещений. В данной модификации запрещаются переходы в состояния, встречавшиеся не более чем k шагов назад вдоль текущей траектории. Ограничение направлено на уменьшение заикливания поиска, возникающего из-за ошибок эвристической функции. При малых значениях k устраняются только непосредственные возвраты и короткие циклы, тогда как большие значения k позволяют подавлять более длинные циклы, однако могут приводить к отсечению полезных переходов. Во всех проведённых экспериментах использовалось значение $k = 10$, которое обеспечивало наилучший баланс между подавлением циклов и сохранением полезных переходов.

ЛП-БП+П — лучевой поиск с локальным запретом повторных посещений и повторными попытками поиска. Если поиск с использованием **ЛП-БП** достигает максимальной глубины без нахождения решения, состояния на последних n уровнях дерева поиска добавляются в множество запрещённых состояний. После этого поиск возвращается к наиболее раннему уровню дерева, содержащему запрещённые состояния, соответствующая часть дерева пересчитывается, и поиск продолжается с обновлёнными ограничениями.

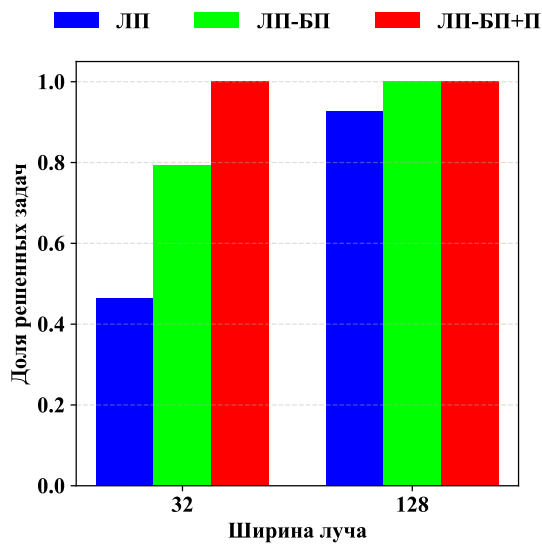
В отличие от полного перезапуска, такой подход позволяет повторно использовать уже построенные части дерева поиска, что обеспечивает приблизительно двукратное ускорение повторных попыток по сравнению с запуском поиска с нуля. В проведённых экспериментах использовалось значение $n = 20$: оно оказалось достаточным для выхода из тупиковых областей поиска и при этом не приводило к избыточному пересчёту дерева.

Результаты

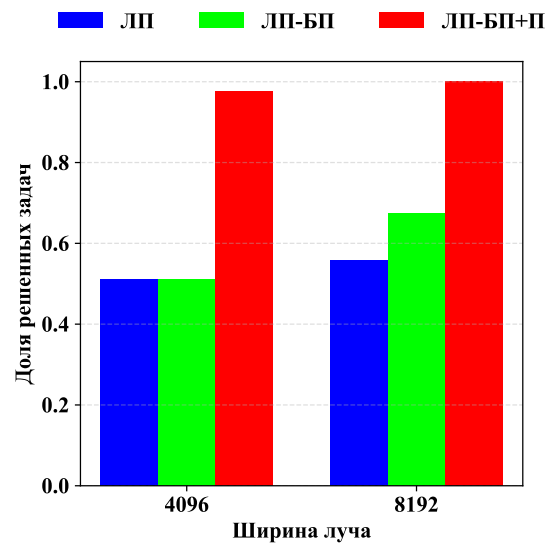
Доля успешно решённых задач в зависимости от ширины луча W и используемой модификации лучевого поиска представлена на рис. 13а, 13б и 13с для кубиков $3 \times 3 \times 3$, $4 \times 4 \times 4$ и $5 \times 5 \times 5$ соответственно. Каждый столбец на графиках соответствует одной из рассматриваемых модификаций алгоритма поиска.

Выводы

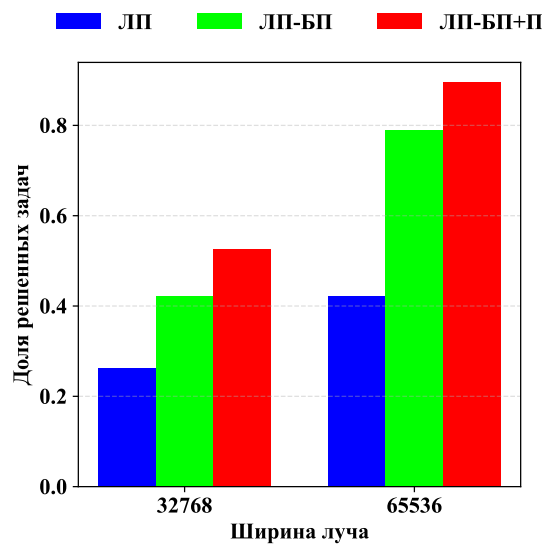
В ходе экспериментов, было выяснено, что введение локального запрета повторных посещений (**ЛП-БП**) даёт ощутимый прирост качества уже при малой ширине луча: для кубика $3 \times 3 \times 3$ при $W = 32$ доля решённых задач возрастает с 0.46 (**ЛП**) до 0.80 (**ЛП-БП**), а добавление повторных попыток поиска (**ЛП-БП+П**) увеличивает её до 1.00 (рис. 13а). Для кубика $4 \times 4 \times 4$ при $W = 4096$ аналогичный прирост — с 0.51 (**ЛП** и **ЛП-БП**) до 0.98 (**ЛП-БП+П**); при $W = 8192$ — с 0.56 до 0.67 и до 1.00 соответственно (рис. 13б). Для кубика $5 \times 5 \times 5$ при $W = 65536$ доля решённых задач возрастает с 0.42 (**ЛП**) до 0.79 (**ЛП-БП**) и до 0.89 (**ЛП-БП+П**) (рис. 13с). Существенно, что повторное использование ранее построенного дерева поиска в **ЛП-БП+П** обеспечивает



(a) Кубик $3 \times 3 \times 3$ (82 задачи).



(b) Кубик $4 \times 4 \times 4$ (43 задачи).



(c) Кубик $5 \times 5 \times 5$ (19 задач).

Рис. 13: Доля решённых задач в зависимости от ширины луча W и модификации лучевого поиска.

примерно двукратное ускорение повторных попыток по сравнению с полным перезапуском, что позволяет применять метод даже при ограниченном вычислительном бюджете.

4.4. Сводные выводы по экспериментам

На основании результатов проведённых экспериментов можно сформулировать следующие наблюдения. Среди трёх исследованных факторов

— способа генерации обучающей выборки, архитектуры нейросетевой модели и модификаций алгоритма поиска — наибольшее практическое влияние оказывают модификации алгоритма поиска (ЛП-БП+П), которые напрямую определяют эффективность исследования пространства состояний.

Выбор архитектуры нейросети при фиксированном датасете оказывает менее выраженное влияние на итоговую долю решённых задач; при этом использование более сложных моделей сопровождается существенным увеличением времени инференса, что ограничивает их практическую применимость в рамках эвристического поиска. Таким образом, использование MLP в качестве архитектуры остаётся единственным практичным вариантом.

Отдельно следует отметить влияние обучающей выборки: использование большого консистентного датасета (БД) приводит к более устойчивой и согласованной оценке расстояния, что проявляется в повышении стабильности работы поиска и снижении чувствительности к ширине луча.

4.5. Сравнение итогового метода с базовым подходом

По результатам проведённых экспериментов был сформирован итоговый метод, включающий:

- лучевой поиск с локальным запретом повторных посещений и повторными попытками поиска (ЛП-БП+П);
- многослойный перцептрон (MLP) в качестве архитектуры эвристической функции;
- обучение на одном большом датасете с консистентными метками (БД).

В качестве базового подхода рассматривается комбинация стандартного beam search без дополнительных модификаций, MLP-эвристики и динамически генерируемых случайных блужданий без обеспечения консистентности меток.

Кубик	Ширина луча	Итоговый метод	Базовый подход
$3 \times 3 \times 3$	2^7	1.00	0.01
$4 \times 4 \times 4$	2^{13}	1.00	0.53
$5 \times 5 \times 5$	2^{16}	0.89	0.32

Таблица 3: Сравнение доли решённых задач между итоговым методом и базовым подходом.

Кубик	Ширина луча	Итоговый метод	Базовый подход
$3 \times 3 \times 3$	2^7	25.00	23.00
$4 \times 4 \times 4$	2^{13}	80.17	76.17
$5 \times 5 \times 5$	2^{16}	149.83	140.17

Таблица 4: Сравнение средней длины найденных решений на пересечении множеств успешно решённых задач.

Результаты

Выводы

Полученные результаты (табл. 3) показывают, что предложенный метод существенно превосходит базовый подход по доле решённых задач для всех рассмотренных размеров кубика. Наиболее выраженный прирост наблюдается для кубика $3 \times 3 \times 3$, где доля решённых задач увеличивается с 0.01 до 1.00 при одинаковой ширине луча. Для кубиков $4 \times 4 \times 4$ и $5 \times 5 \times 5$ также наблюдается значительное улучшение качества поиска.

При этом средняя длина найденных решений (табл. 4) для итогового метода оказывается немного больше, чем у базового подхода. Однако данное ухудшение является сравнительно небольшим по сравнению с существенным ростом доли успешно решённых задач и связано с тем, что модифицированный поиск ориентирован прежде всего на повышение устойчивости и вероятности нахождения решения при ограниченной ширине луча.

Таким образом, сочетание консистентного обучающего датасета, устойчивой эвристической функции и модифицированного лучевого поиска позволяет существенно повысить эффективность поиска на графах Кэли кубиков Рубика без необходимости экстремального увеличения ширины луча и вычислительных затрат.

Заключение

В данной работе исследована задача поиска кратчайшего пути на графах Кэли конечных групп на примере кубиков Рубика размера $N \times N \times N$ при $N \in \{3, 4, 5\}$. Задача является NP-трудной в общем случае, а при $N \geq 4$ размер пространства состояний (порядка 10^{45} и выше) исключает применение как точных алгоритмов перебора (BFS, IDA*), так и методов с предварительно вычисленными таблицами эвристик (pattern databases).

В ходе работы были достигнуты следующие результаты:

1. Проведён обзор существующих подходов к решению задачи — методов на основе теории групп, эвристического поиска с предварительно вычисленными эвристиками и методов эвристического поиска с применением машинного обучения (DeepCubeA, EfficientCube, CayleyPy). Проанализированы причины их ограниченной применимости при $N \geq 4$: экспоненциальный рост объёма таблиц, недопустимые требования к памяти, деградация качества при росте ширины луча.
2. Реализован генератор обучающих данных на основе случайных блужданий по графу Кэли трёх модификаций — стандартных блужданий без немедленных возвратов, блужданий с глобальным запретом повторных посещений и генерация одного большого датасета с глобальным запретом повторных посещений. Экспериментально показано, что консистентность меток расстояния (наличие одной уникальной метки для каждого состояния из датасета) является ключевым фактором качества обучения: использование фиксированного датасета с глобальным запретом повторных посещений (БД) повышает долю решённых задач для кубика $3 \times 3 \times 3$ с 0.40 до 1.00 при ширине луча $W = 1024$ по сравнению с базовым вариантом без консистентности.
3. Реализованы и сравнены три архитектуры нейросетевой функции оценки: MLP, GNN (RGCN) и SA (Self-Attention). Установлено, что GNN и SA дают лишь незначительный прирост качества при существенно большем времени инференса — до 3.3 раз для SA на кубике $4 \times 4 \times 4$.

На основании этого в качестве основной архитектуры выбран MLP как оптимальный по соотношению качества и скорости.

4. Предложены и исследованы две модификации базового лучевого поиска: локальный запрет повторных посещений (ЛП-БП) и повторные попытки поиска с переиспользованием дерева (ЛП-БП+П). Совместное применение этих модификаций позволило достичь доли решённых задач 1.00 для кубиков $3 \times 3 \times 3$ и $4 \times 4 \times 4$ и 0.89 для кубика $5 \times 5 \times 5$ при намного меньшей ширине луча, чем требует подход со стандартным лучевым поиском.

Итоговый метод — сочетание MLP-эвристики, датасета БД и алгоритма ЛП-БП+П — по сравнению с базовым подходом увеличивает долю решённых задач с 0.01 до 1.00 для кубика $3 \times 3 \times 3$, с 0.53 до 1.00 для $4 \times 4 \times 4$ и с 0.32 до 0.89 для $5 \times 5 \times 5$, при этом незначительно ухудшая среднюю длину найденных решений.

Направлениями дальнейших исследований являются масштабирование метода на кубики $N \geq 6$, разработка более информативных представлений состояния для нейросетевой эвристики, а также исследование адаптивных стратегий управления шириной луча и числом повторных попыток в зависимости от сложности задачи.

Список литературы

- [1] Pivtoraiko, M. Differentially constrained mobile robot motion planning in state lattices / M. Pivtoraiko, R. A. Knepper, A. Kelly // Journal of Field Robotics. – 2009. – Т. 26, № 3. – С. 308–333.
- [2] Even S., Goldreich O. The minimum-length generator sequence problem is NP-hard // Journal of Algorithms. — 1981. — Т. 2, № 3. — С. 311–313.
- [3] Demaine, E. D. Algorithms for solving Rubik’s cubes / E. D. Demaine, M. L. Demaine, S. Eisenstat, A. Lubiw, A. Winslow // Proceedings of the 19th Annual European Symposium on Algorithms (ESA 2011). – Berlin : Springer, 2011. – С. 689–700.
- [4] Costanzo, M. Genetic landscape of a cell / M. Costanzo, A. Baryshnikova, J. Bellay [и др.] // Science. – 2010. – Т. 327, № 5964. – С. 425–431.
- [5] Linton S. GAP: Groups, Algorithms, Programming // ACM Communications in Computer Algebra. — 2007. — Т. 41, № 2. — С. 108–109.
- [6] Symmetric group 4; Cayley graph as half-turns of 2×2×2 Rubik’s cube [Электронный ресурс] // Wikimedia Commons. — URL: https://commons.wikimedia.org/wiki/File:Symmetric_group_4;_Cayley_graph_as_half-turns_of_2%C3%972%C3%972_Rubik%27s_cube.svg (дата обращения: 24.05.2026).
- [7] Agostinelli F., McAleer S., Shmakov A., Baldi P. Solving the Rubik’s Cube with deep reinforcement learning and search // Nature Machine Intelligence. — 2019. — Т. 1, № 8. — С. 356–363.
- [8] Takano K. Self-Supervision is All You Need for Solving Rubik’s Cube // Transactions on Machine Learning Research. — 2023. — № 11.
- [9] Holbrook R., Reade W., Howard A. Santa 2023 — The Polytope Permutation Puzzle [Электронный ресурс] // Kaggle Competition. — 2023. — URL: <https://kaggle.com/competitions/santa-2023> (дата обращения: 17.03.2026).

- [10] Chervov A., Khoruzhii K., Bukhal N. [и др.] A machine learning approach that beats Rubik's cubes // *Advances in Neural Information Processing Systems (NeurIPS 2025)*. — 2025. — Т. 38. — URL: <https://neurips.cc/virtual/2025/poster/120075> (дата обращения: 17.03.2026).
- [11] Touvron, H. ResMLP: feedforward networks for image classification with data-efficient training / H. Touvron, P. Bojanowski, M. Caron, M. Cord, A. El-Nouby, E. Grave, G. Izacard, A. Joulin, G. Synnaeve, J. Verbeek, H. Jégou // *IEEE Transactions on Pattern Analysis and Machine Intelligence*. — 2023. — Т. 45, № 4. — С. 5314–5321.
- [12] McAleer, S. Solving the Rubik's cube without human knowledge / S. McAleer, F. Agostinelli, A. Shmakov, P. Baldi // *arXiv:1805.07470*. — 2018.
- [13] Thistlethwaite M. B. *The 45–52 Move Strategy*. — London, 1981. — (Cube-Lovers, CL VIII).
- [14] Kociemba H. Close to God's algorithm // *Cubism For Fun*. — 1992. — № 28. — С. 10–13.
- [15] Korf R. E. Finding optimal solutions to Rubik's Cube using pattern databases // *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. — Providence, RI, 1997. — С. 700–705.
- [16] Tiapkin D. [и др.] Learning Shortest Paths with Generative Flow Networks [Электронный ресурс]. — 2025. — *arXiv:2603.01786*. — URL: <https://arxiv.org/abs/2603.01786> (дата обращения: 27.03.2026).
- [17] Bengio, E. Flow network based generative models for non-iterative diverse candidate generation / E. Bengio, M. Jain, M. Korablyov, D. Precup, Y. Bengio // *Advances in Neural Information Processing Systems*. — 2021. — Т. 34. — С. 27381–27394.
- [18] Rokicki, T. The diameter of the Rubik's cube group is twenty / T. Rokicki, H. Kociemba, M. Davidson, J. Dethridge // *SIAM Review*. — 2014. — Т. 56, № 4. — С. 645–670.

- [19] Moore, E. F. The shortest path through a maze / E. F. Moore // Proc. of the International Symposium on the Theory of Switching. – Cambridge : Harvard University Press, 1959. – С. 285–292.
- [20] Pearl, J. Heuristics: intelligent search strategies for computer problem solving / J. Pearl. – Reading, MA : Addison-Wesley, 1984. – 382 с.
- [21] Hart, P. E. A formal basis for the heuristic determination of minimum cost paths / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. – 1968. – Т. 4, № 2. – С. 100–107.
- [22] Korf, R. E. Depth-first iterative-deepening: An optimal admissible tree search / R. E. Korf // Artificial Intelligence. – 1985. – Т. 27, № 1. – С. 97–109.
- [23] Lowerre, B. T. The HARPY speech recognition system : PhD thesis / B. T. Lowerre. – Pittsburgh : Carnegie Mellon University, 1976. – 262 с.
- [24] Rumelhart, D. E. Learning representations by back-propagating errors / D. E. Rumelhart, G. E. Hinton, R. J. Williams // Nature. – 1986. – Т. 323, № 6088. – С. 533–536.
- [25] Amanatullah. Unleashing the Power of Multi-Layer Perceptron: Decoding the Significance of Weights and Biases [Электронный ресурс] / Amanatullah // Medium. – 2023. – URL: <https://medium.com/%40amanatulla1606/unleashing-the-power-of-multi-layer-perceptron-decoding-the-significance-of-weights-and-biases-9ebdc4df0f33> (дата обращения: 24.05.2026).
- [26] Scarselli, F. The graph neural network model / F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, G. Monfardini // IEEE Transactions on Neural Networks. – 2009. – Т. 20, № 1. – С. 61–80.
- [27] Li, S. Property graph representation learning for node classification / S. Li, N. A. Zaidi, M. Du, Z. Zhou, H. Zhang, G. Li // Knowledge and Information Systems. – 2024. – Т. 66, № 1. – С. 237–265.

- [28] Vaswani, A. Attention is all you need / A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin // Advances in Neural Information Processing Systems. – 2017. – T. 30.
- [29] Schlichtkrull, M. Modeling relational data with graph convolutional networks / M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling // The Semantic Web. ESWC 2018. – Cham : Springer, 2018. – C. 593–607.
- [30] Meister, C. If beam search is the answer, what was the question? / C. Meister, R. Cotterell, T. Vieira // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Online : Association for Computational Linguistics, 2020. – C. 2173–2185.